

Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática
Aprendizagem e Decisão Inteligentes
Ano Letivo 2025/2026

Relatório de Resultados

Análise de Dados e Modelação com KNIME

Spotify Tracks & Consumo Energético

Grupo 11

Marco Sèvegrand A106807
Francisco Martins A106902
Nuno Rebelo Axxxxxxx
Marco Ferreira Axxxxxxx

13 de maio de 2026

Conteúdo

1	Introdução	1
	Parte I — Tarefa 1: Dataset de Grupo (Spotify Tracks)	2
2	Domínio e objetivos	2
2.1	Domínio analisado	2
2.2	Objetivos	2
3	Metodologia CRISP-DM	3
3.1	Compreensão do negócio	3
3.2	Compreensão dos dados	3
3.3	Preparação dos dados	5
3.3.1	Remoção de duplicados	5
3.3.2	Sanitização e engenharia de atributos	6
3.3.3	Imputação de valores em falta	6
3.3.4	Tratamento de <i>outliers</i>	6
3.3.5	Seleção de variáveis e separação de ramos	7
3.4	Modelação	7
3.4.1	<i>Clustering</i> de perfis acústicos	7
3.4.2	Classificação de género musical	9
3.4.3	Regressão da popularidade	12
3.5	Avaliação	13
4	Resultados e análise crítica	14
4.1	Leitura crítica do <i>clustering</i>	14
4.2	Leitura crítica da classificação de género	14
4.3	Relação entre <i>clustering</i> e classificação	15
4.4	Leitura crítica da regressão	15
4.5	Limitações	16
	Parte II — Tarefa 2: Dataset Atribuído (Consumo Energético)	20
5	Definição da metodologia	20
5.1	Visão geral do workflow desenvolvido no KNIME	20
5.2	Exploração	21
5.3	Transformação	22
6	Modelação e Avaliação	23
6.1	Random Forest	24
6.2	Gradient Boosted Trees	25
6.3	Multilayer Perceptron	26
6.4	Keras	28
7	Conclusão	31

1 Introdução

O presente relatório documenta o trabalho desenvolvido no âmbito da unidade curricular de Aprendizagem e Decisão Inteligentes, correspondente à componente prática de grupo. O objetivo central consistiu em conceber, implementar e avaliar modelos de aprendizagem automática sobre dois conjuntos de dados distintos, recorrendo à plataforma KNIME como ambiente de desenvolvimento e experimentação.

O trabalho estrutura-se em duas tarefas independentes, conforme definido no enunciado. A **Tarefa 1** incide sobre um **dataset de grupo** — um conjunto de dados sobre faixas musicais do Spotify, gerado sinteticamente para simular um cenário realista de preparação de dados. Nesta tarefa, a metodologia adotada foi o **CRISP-DM**, aplicando-se técnicas de *clustering* para identificação de perfis acústicos, modelos de classificação para previsão do género musical e modelos de regressão para previsão da popularidade das faixas. A **Tarefa 2** é dedicada a um **dataset atribuído** — um conjunto de dados sobre consumo energético e variáveis meteorológicas, distribuído pela equipa docente através do Blackboard. Nesta tarefa, seguiu-se a metodologia **SEMMA**, desenvolvendo-se modelos de classificação e de regressão com base nas relações entre variáveis climáticas e os padrões de consumo registados.

Cada tarefa é descrita em parte própria do relatório, com a respetiva fundamentação metodológica, descrição das fases de análise e modelação, e discussão crítica dos resultados. As figuras e tabelas mais extensas são remetidas para a secção de Anexos, sendo referenciadas ao longo do texto.

Parte I — Tarefa 1: Dataset de Grupo (Spotify Tracks)

2 Domínio e objetivos

2.1 Domínio analisado

O dataset `spotify_tracks.csv` foi gerado por um *script* Python especificamente concebido para simular um conjunto de dados realista sobre faixas musicais da plataforma Spotify. O ficheiro contém **100 500 registos** e **21 colunas**, cobrindo o período temporal de **2000 a 2024** e **20 géneros** musicais distintos. Cada registo representa uma faixa, caracterizada por atributos de identificação, variáveis de contexto editorial e um conjunto de *features* acústicas extraídas da API do Spotify.

O domínio analisado é o da análise musical e do consumo digital. As *features* acústicas — como `danceability`, `energy`, `loudness` e `acousticness` — capturam propriedades sonoras objetivas, enquanto variáveis como `genre`, `release_year` e `explicit` fornecem contexto editorial. O target principal é `popularity`, uma pontuação numérica entre 0 e 100.

O gerador não produziu um *dataset* limpo; pelo contrário, introduziu deliberadamente problemas de qualidade comuns em cenários reais de ciência de dados. Foram injetados valores em falta, *outliers* numéricos em várias colunas e 500 linhas duplicadas. Existe ainda um ficheiro auxiliar, `spotify_tracks_errors_log.csv`, que regista a localização exata de cada erro introduzido e serviu como referência para validação das etapas de preparação. A Tabela ?? (Anexo ??) sintetiza a estrutura do dataset.

2.2 Objetivos

Os objetivos definidos para esta tarefa foram:

- garantir uma preparação robusta dos dados, com tratamento de duplicados, valores em falta, *outliers* e criação de atributos derivados;
- identificar perfis acústicos através de *k-means* e interpretar a composição dos *clusters* obtidos;
- treinar modelos de classificação para prever o género musical (`genre`, 20 classes) a partir das *features* acústicas, relacionando esta abordagem supervisionada com a segmentação não supervisionada do *clustering*;
- treinar modelos de regressão para prever `popularity` a partir das variáveis acústicas, de contexto e do género musical (codificado em *one-hot*);
- produzir uma análise crítica dos resultados, com foco na relação entre *clustering* e classificação, na contribuição do género para a regressão e no desempenho preditivo cruzado das três vertentes.

3 Metodologia CRISP-DM

A metodologia CRISP-DM (*Cross-Industry Standard Process for Data Mining*) organiza o processo de análise de dados em seis fases iterativas: compreensão do negócio, compreensão dos dados, preparação dos dados, modelação, avaliação e *deployment*. Esta metodologia orientou a estrutura do *workflow* KNIME e a organização desta secção do relatório.

3.1 Compreensão do negócio

O objetivo analítico é duplo: caracterizar faixas musicais através de perfis acústicos (segmentação) e prever a popularidade com base nas propriedades sonoras e de contexto (regressão). Na prática, estes objetivos traduzem-se em casos de uso como segmentação de catálogo, recomendação musical e análise do grau em que os atributos de áudio explicam o sucesso comercial de uma faixa.

3.2 Compreensão dos dados

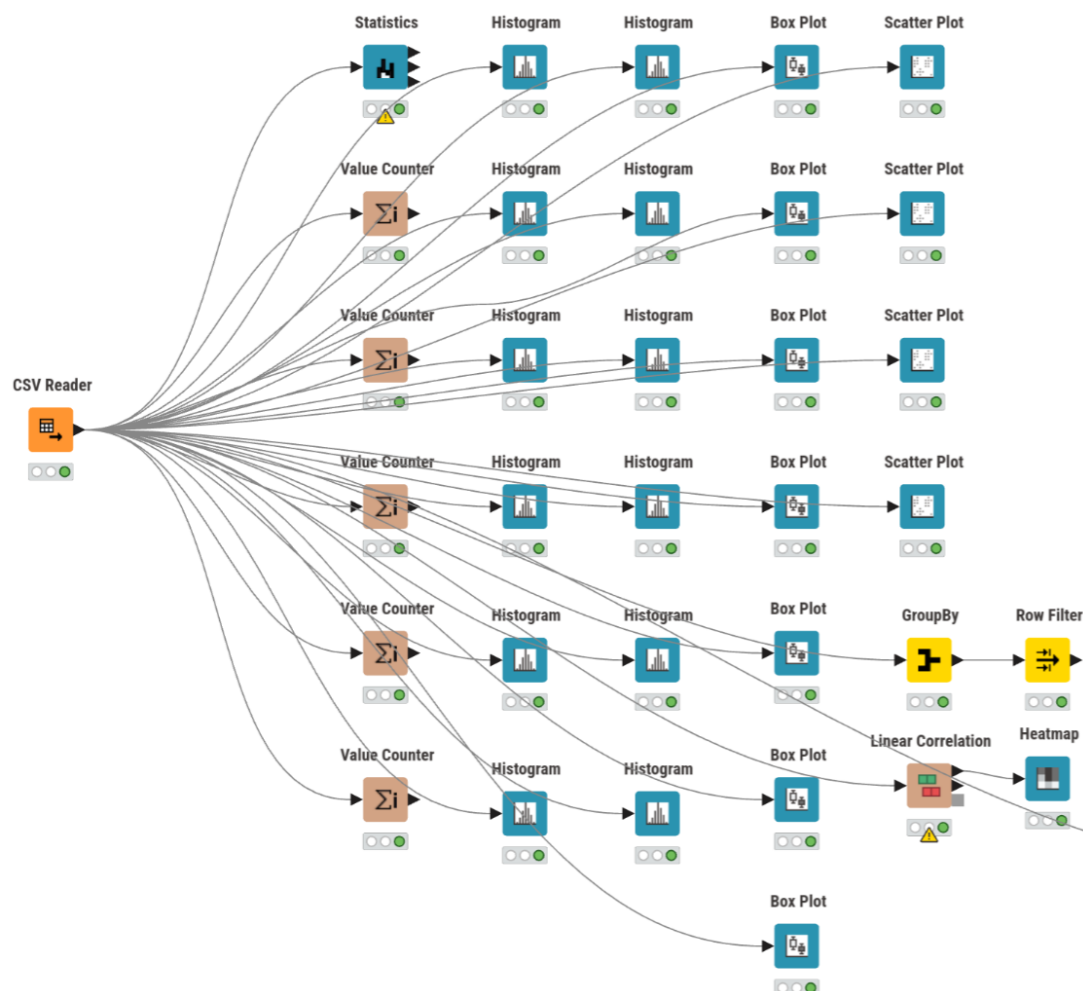


Figura 1: Workflow da fase de Compreensão dos Dados

O bloco de *Data Understanding* do *workflow* foi implementado com 31 nós KNIME, organizados para cobrir todas as dimensões relevantes da qualidade dos dados. A leitura inicial, através

do nó **CSV Reader** (#2), confirmou a importação de 100 500 linhas e 21 colunas, com as *features* de áudio corretamente interpretadas como numéricas.

O nó **Statistics** (#3) foi configurado para 12 colunas numéricas — **release_year**, **popularity**, **duration_ms**, **danceability**, **energy**, **loudness**, **speechiness**, **acousticness**, **instrumentalness**, **liveness**, **valence** e **tempo** — e revelou, de forma quantitativa, os problemas centrais do dataset. As *features* acústicas apresentam cerca de 3 000 valores em falta por coluna (por exemplo: 3013 em **danceability**, 3021 em **speechiness** e 3026 em **tempo**). Em paralelo, várias colunas limitadas teoricamente ao intervalo $[0, 1]$ exibem violações claras de domínio, como **danceability** com mínimo -0.299 e máximo 1.497 , **energy** com mínimo -0.298 e máximo 1.498 , e **valence** com mínimo -0.300 e máximo 1.497 . Detetaram-se ainda extremos artificiais em **loudness** (até -89.95 dB) e **tempo** (até 419.95 BPM), consistentes com a injeção deliberada de *outliers* prevista no gerador.

A componente categórica foi examinada com cinco nós **Value Counter**, cobrindo **genre** (20 géneros confirmados), **explicit**, **mode**, **key** e **time_signature**. A exploração univariada assentou em 12 histogramas e 7 *boxplots* (Figuras 2 e 2)

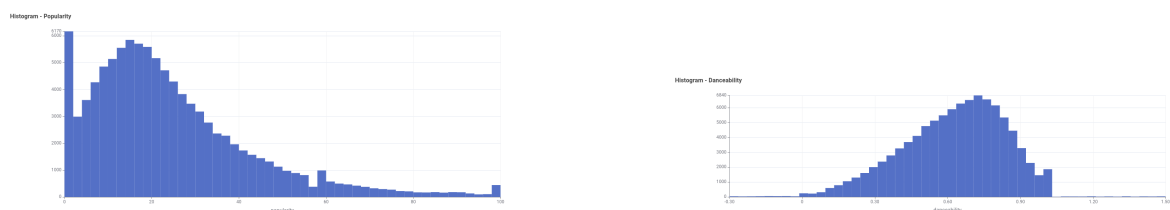


Figura 2: Distribuições de Popularity e Danceability (exemplos)

, que documentaram as distribuições das variáveis contínuas. As relações bivariadas foram analisadas com 4 gráficos de dispersão — pares **energy-loudness**, **energy-acousticness**, **danceability-valence** e **release_year-popularity** — e uma matriz de correlação linear sobre 15 colunas (Figura 3)

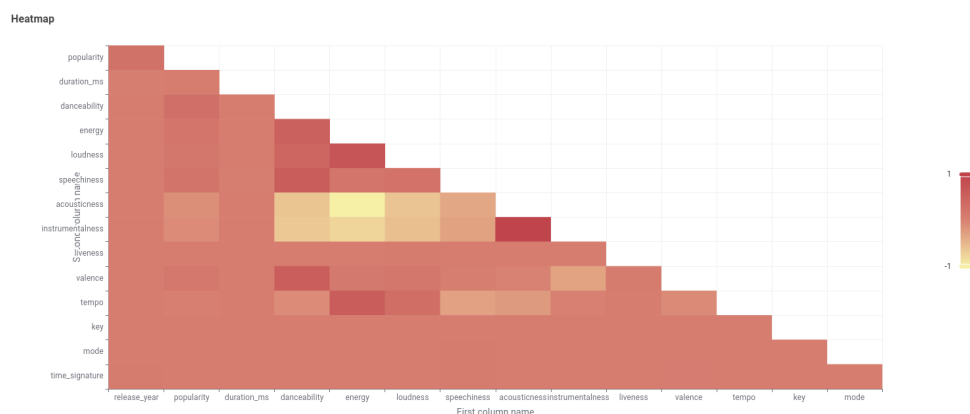


Figura 3: Matriz de correlação linear sobre 15 colunas

A deteção de duplicados foi realizada através do par **GroupBy** (#40) + **Row Filter** (#39), que isolou 500 registos repetidos, confirmando a injeção de duplicados prevista pelo gerador.

Em síntese, esta fase demonstrou que o *dataset* continha, de forma inequívoca, os quatro

tipos de problemas que a preparação subsequente teria de resolver: valores em falta, violações de domínio, *outliers* e duplicados. A Tabela 1 resume a evidência recolhida.

Tabela 1: Problemas de qualidade evidenciados na fase de compreensão dos dados		
Problema	Evidência no workflow	Consequência
Valores em falta	Cerca de 3 000 missings por <i>feature</i> acústica; p.ex. 3 013 em <i>danceability</i> , 3 021 em <i>speechiness</i> , 3 026 em <i>tempo</i>	Imputação em Missing Value
Valores fora de domínio	<i>danceability</i> , <i>energy</i> e <i>valence</i> com mínimos negativos e máximos acima de 1; <i>popularity</i> validada em [0, 100]	Sanitização em Expression
Extremos artificiais	<i>tempo</i> até 419.95, <i>loudness</i> até -89.95 dB, <i>duration_ms</i> com extremos fora do perfil central	Tratamento em Numeric Outliers
Duplicados	GroupBy e Row Filter isolam exatamente 500 casos	Remoção em Duplicate Row Filter

3.3 Preparação dos dados

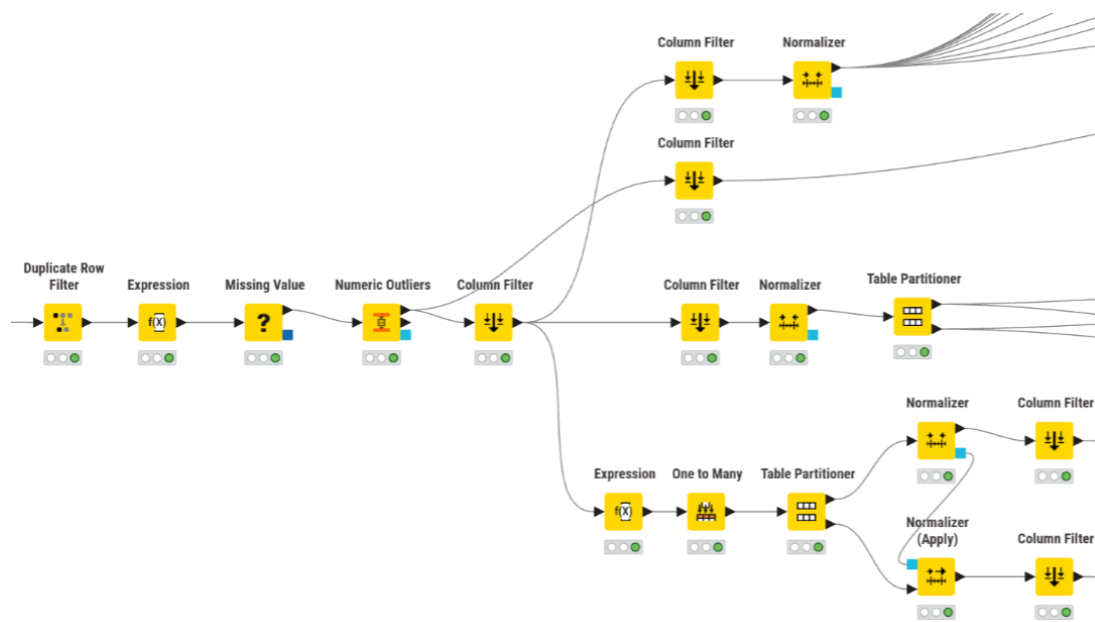


Figura 4: Workflow da fase de Preparação dos Dados

A preparação dos dados foi implementada como um pipeline sequencial com cinco etapas principais, executadas antes da separação entre os ramos de *clustering* e regressão. A Figura ?? (Anexo ??) ilustra a configuração do pipeline no KNIME.

3.3.1 Remoção de duplicados

A primeira operação do pipeline é o nó **Duplicate Row Filter** (#41), que compara todas as colunas exceto *track_id*, remove as linhas repetidas e conserva a primeira ocorrência. O resultado

confirma a passagem de 100 500 para 100 000 linhas, eliminando os 500 duplicados.

3.3.2 Sanitização e engenharia de atributos

O nó **Expression** (#42) concentra num único ponto oito transformações que, num *workflow* menos compacto, estariam dispersas por múltiplos nós. As operações realizadas foram:

- conversão de valores fora do domínio $[0, 100]$ de **popularity** para *missing*;
- conversão de violações do intervalo $[0, 1]$ nas *features* de áudio (**danceability**, **energy**, **speechiness**, **acousticness**, **instrumentalness**, **liveness** e **valence**) para *missing*;
- conversão de valores inválidos de **key**, **mode** e **time_signature** para *missing*;
- codificação binária da variável **explicit** ($\text{True} \rightarrow 1$, $\text{False} \rightarrow 0$);
- criação do atributo derivado **duration_min** ($= \text{duration_ms} / 60\,000$), mais interpretável do que milissegundos;
- criação da classe auxiliar **popularity_class** com três níveis (Baixa, Média, Alta) para estratificação.

Esta etapa é metodologicamente relevante porque, em vez de eliminar precocemente as linhas problemáticas, opta por converter violações de domínio em valores em falta, delegando a decisão para o nó seguinte.

3.3.3 Imputação de valores em falta

O nó **Missing Value** (#43) recebe tanto os valores em falta originais do CSV como os gerados pelo **Expression**. A estratégia de imputação adotada não é uniforme, refletindo uma escolha ponderada por coluna:

- **Média:** **danceability**, **valence**.
- **Mediana:** **energy**, **speechiness**, **acousticness**, **instrumentalness**, **liveness**, **loudness**, **tempo**, **duration_ms**.
- **Remover linha:** **popularity** (target).

Após esta fase, o *dataset* regista 99 353 linhas (menos 647 face às 100 000 anteriores), correspondendo às linhas cujo target **popularity** era inválido e não foi imputado.

3.3.4 Tratamento de *outliers*

Os *outliers* contínuos são tratados no nó **Numeric Outliers** (#45), que intervém sobre três colunas — **duration_ms**, **loudness** e **tempo** — utilizando o critério IQR com fator 3.0 e substituindo os valores extremos pelo limite admissível mais próximo. Esta abordagem preserva as linhas afetadas, evitando perda adicional de dados.

3.3.5 Seleção de variáveis e separação de ramos

O nó **Column Filter** (#47) reduz o *dataset* a 15 colunas, removendo identificadores (`track_id`, `track_name`, `artist_name`, `album_name`), colunas redundantes (`duration_ms`, substituída por `duration_min`), e os atributos `key` e `time_signature`. A coluna `genre`, ao contrário da versão anterior do *workflow*, é preservada em todos os ramos. A base comum resultante contém 99 353 linhas e 15 colunas, a partir da qual se abrem três ramos:

- **Ramo de *clustering***: reduzido a 10 *features* acústicas normalizadas (Min-Max para $[0, 1]$) — sem género, que é reintroduzido apenas no pós-processamento para interpretação (**Joiner** (#136) por `RowID`);
- **Ramo de classificação**: filtra 11 colunas — as 10 *features* acústicas normalizadas mais o target `genre` — e divide-se em dois sub-ramos com partições independentes (70/30 para a rede neuronal, 80/20 para a *Random Forest*), ambas estratificadas por género com `seed=1`;
- **Ramo de regressão**: o nó **Expression** (#229) cria a variável auxiliar `popularity_class` (Baixa/Media/Alta) para estratificação. O nó **One to Many** (#221) codifica `genre` em 20 colunas binárias (*one-hot encoding*), expandindo o espaço de *features* para 34 colunas. Segue-se a partição treino/teste (80/20, estratificada por `popularity_class`, `seed=1`), normalização Min-Max sobre as *features* e remoção de `popularity_class` no treino e no teste. As *features* finais de regressão são 33 preditores — `release_year`, `explicit`, as 10 *features* de áudio, `mode`, `duration_min` e as 20 dummies de género — com target `popularity`.

3.4 Modelação

3.4.1 *Clustering* de perfis acústicos

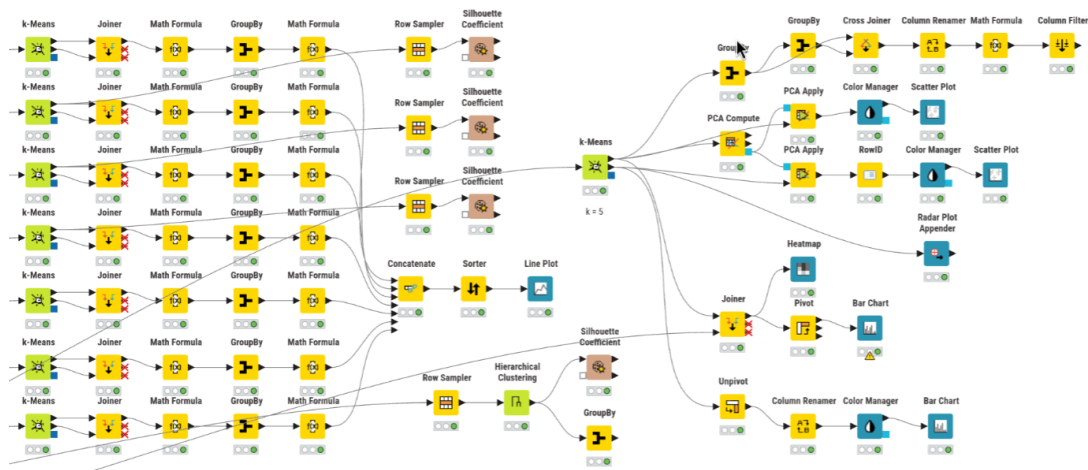


Figura 5: Workflow da fase de Clustering

Método do cotovelo. A seleção do número de *clusters* baseou-se no método do cotovelo. Foram executadas sete experiências de *k-means* para valores de $k \in \{3, 4, 5, 6, 7, 8, 9\}$, com inicialização aleatória, `seed=1` e 99 iterações máximas. Em cada ramo candidato, a soma das distâncias quadradas intra-*cluster* (WCSS) foi calculada manualmente: o nó **Joiner** associa cada ponto ao centróide do respetivo *cluster*, o nó **Math Formula** calcula a distância euclidiana quadrada às

10 coordenadas do centróide, e o nó **GroupBy** agrega os valores. As sete linhas de WCSS foram concatenadas e representadas graficamente (Figura 6)

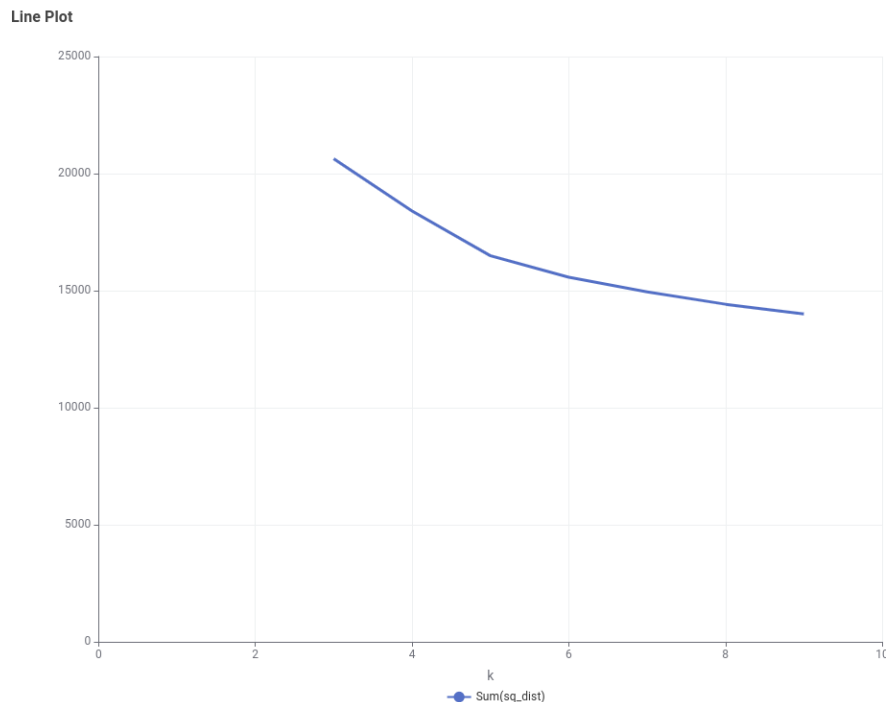


Figura 6: Soma das distâncias quadradas intra-cluster (WCSS) para diferentes valores de k

Modelo final. O modelo de *clustering* adotado fixou $k = 5$, coerente com a leitura da curva do cotovelo. O nó **k-Means** (#141) produz diretamente a coluna **Cluster**, dispensando um *assigner* separado. Os centróides foram transformados para formato longo (**Unpivot**) e visualizados num gráfico de barras agrupado (Figura 7)

, que permite comparar o perfil acústico médio de cada *cluster* nas 10 *features* normalizadas.

Interpretação. A dimensão dos *clusters* foi calculada através de **GroupBy** (contagem por **Cluster**) e **Cross Joiner** (total global), obtendo-se a percentagem relativa de cada grupo. Para interpretação semântica, a coluna **genre** — que não integra o espaço de *features* do *clustering* — foi reintroduzida via **Joiner** por **RowID**, permitindo construir uma matriz **Cluster** $\times$ **Genre** com **Pivot** e um gráfico de barras empilhadas (Figura 17)

Validação. A qualidade do agrupamento foi avaliada com o coeficiente de *silhouette*. O método hierárquico (*Euclidean, linkage complete*) sobre uma amostra de 5 000 linhas produziu um valor médio global de **0.0883**. Para os modelos *k-means*, os valores médios observados foram **0.1802** para $k = 4$, **0.1805** para $k = 5$ e **0.1619** para $k = 6$. Estes resultados mostram que $k = 4$ e $k = 5$ são muito próximos em termos de coesão/separação, ao passo que $k = 6$ introduz maior fragmentação sem ganho correspondente de qualidade. A escolha final de $k = 5$ apoia-se, por isso, na conjugação entre o método do cotovelo e a maior interpretabilidade semântica dos cinco perfis acústicos resultantes.



Figura 7: Perfil acústico médio de cada cluster

3.4.2 Classificação de género musical

A novidade face à versão anterior do *workflow* é a inclusão de um ramo de classificação supervisionada que utiliza as mesmas 10 *features* acústicas que alimentam o *k-means*, mas com o objetivo de prever o género musical (20 classes). Este desenho permite contrastar diretamente a aprendizagem não supervisionada (agrupamento por semelhança acústica, sem conhecimento dos rótulos) com a aprendizagem supervisionada (classificação informada pelo género real), medindo até que ponto os atributos de áudio contêm sinal suficiente para discriminar estilos musicais.

Configuração. O ramo de classificação parte da base comum pós-preparação. O nó **Column Filter** (#220) isola 11 colunas: as 10 *features* acústicas e o target **genre**. A normalização Min-Max $[0, 1]$ é aplicada através do nó **Normalizer** (#218), garantindo que todas as variáveis contribuem em escala comparável.

Foram treinados dois modelos supervisionados:

- **RProp MLP** (#226): rede neuronal *feedforward* com uma camada escondida de 30 neurónios, treinada com o algoritmo *RProp* durante 200 iterações (*seed*=1). A partição é 70/30 (treino/teste), estratificada por género, resultando em 70 000 exemplos de treino e 30 000 de teste.
- **Random Forest** (#235): *ensemble* de 200 árvores de decisão, critério Gini, amostragem de colunas em modo $\sqrt{\cdot}$ (*SquareRoot*), com *seed*=1. A partição é 80/20 (treino/teste), estratificada por género, resultando em 80 000 exemplos de treino e 20 000 de teste.

A escolha de partições diferentes (70/30 vs. 80/20) reflete uma preocupação metodológica: a MLP, por ser um modelo paramétrico com maior propensão ao sobreajustamento, beneficia de um conjunto de teste mais numeroso para validação fiável; a *Random Forest*, enquanto método de *ensemble*, tolera uma proporção de teste ligeiramente menor sem perda de robustez estatística.

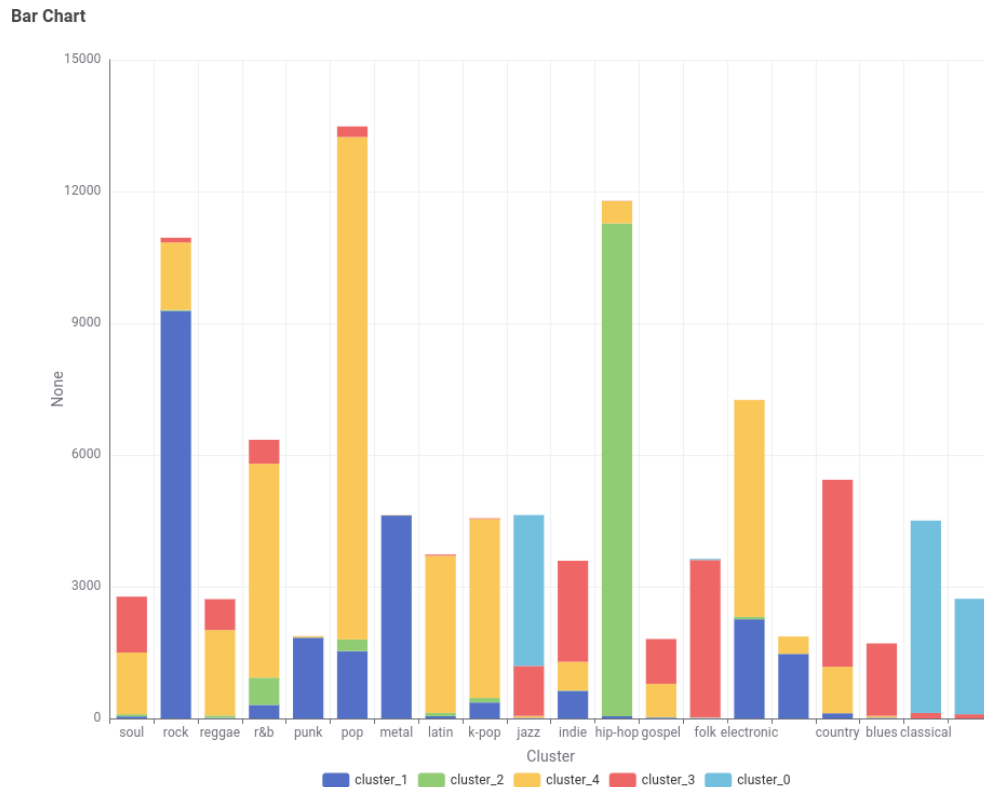


Figura 8: Distribuição dos gêneros musicais por cluster

Para a *Random Forest*, foi ainda realizado um estudo de sensibilidade sobre configurações exportadas, incluindo diferentes profundidades/capacidades do conjunto, critério de divisão (Gini vs. *Information Gain Ratio*) e número de árvores. Este estudo permite avaliar não apenas o desempenho do modelo final, mas também a estabilidade do problema de classificação face a alterações de hiperparâmetros.

Métricas. A avaliação foi realizada através do nó **Scorer**, que calcula a exatidão (*accuracy*), o erro, o número de classificações corretas e incorretas, e o coeficiente Kappa de Cohen (que corrige a exatidão pela probabilidade de acerto aleatório, sendo particularmente informativo em problemas com 20 classes). Nesta fase do trabalho, os resultados exportados da MLP continuam indisponíveis, mas a família *Random Forest* foi analisada em múltiplas configurações. A matriz de confusão visualizada no *Heatmap* permite inspecionar a distribuição dos erros por classe (Figura 10)

A Tabela 2 sintetiza os resultados atualmente disponíveis. A configuração base do *workflow* (Gini, 200 árvores) atinge **68.36%** de exatidão (Kappa = **0.6563**). O melhor resultado exportado corresponde à configuração Gini com 500 árvores, com **68.44%** de exatidão e Kappa de **0.6571**. O ganho absoluto de apenas 0.08 pontos percentuais mostra que o problema satura rapidamente: aumentar a complexidade do conjunto melhora pouco porque a limitação principal parece residir na sobreposição entre classes, e não na capacidade do algoritmo.

Relação com o clustering. A justaposição do ramo de classificação com o de *clustering* é metodologicamente reveladora. Ambos operam sobre exatamente as mesmas 10 *features* acústicas, mas com filosofias opostas:

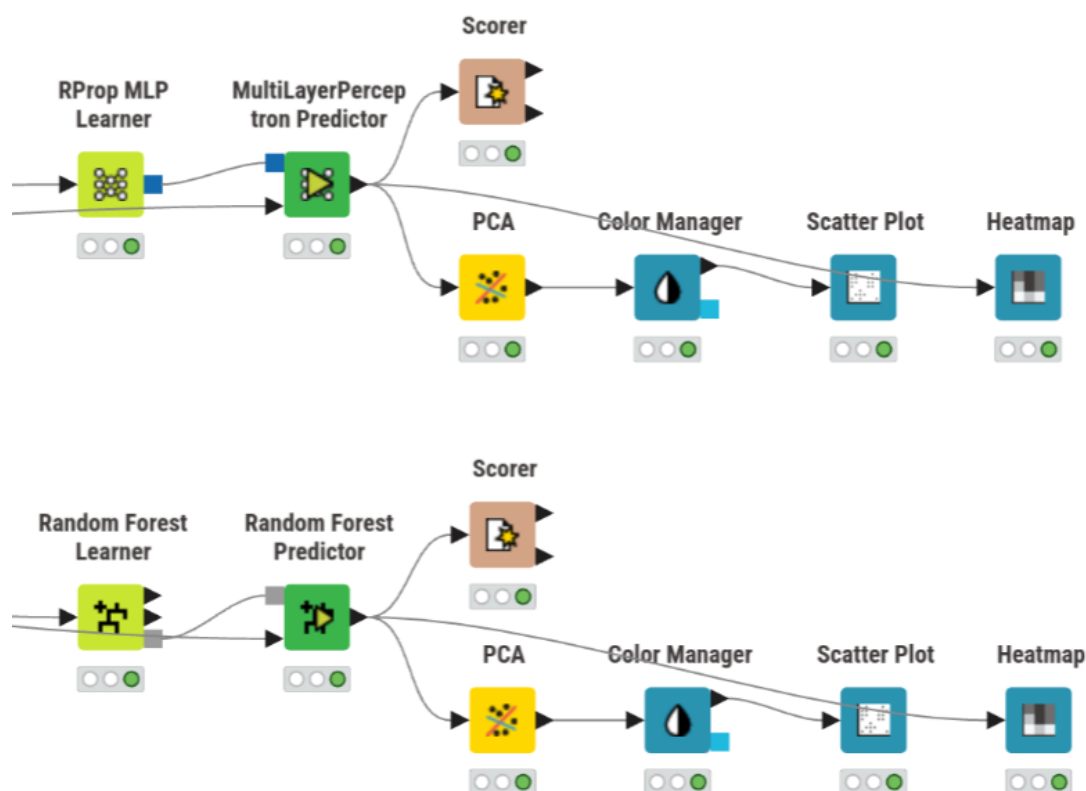


Figura 9: Workflow da fase de Classificação

Tabela 2: Métricas de classificação — Previsão de gênero musical

Modelo	Exatidão	Erro	Corretas	Incorretas
RProp MLP (70/30)	<i>pendente</i>	<i>pendente</i>	<i>pendente</i>	<i>pendente</i>
<i>Random Forest</i> base (Gini, 200 árvores)	0.6836	0.3164	13 672	6 328
<i>Random Forest</i> melhor configuração (Gini, 500 árvores)	0.6844	0.3156	13 688	6 312

- O *k-means* (não supervisionado) agrupa faixas por proximidade no espaço acústico, sem qualquer conhecimento dos gêneros reais; a interpretação dos grupos é feita *a posteriori* através da distribuição dos gêneros em cada *cluster*.
- A classificação (supervisionada) aprende a fronteira de decisão a partir dos rótulos de gênero, medindo diretamente a separabilidade acústica das 20 classes.

O coeficiente de *silhouette* entre 0.1619 e 0.1805 obtido no *clustering* sugere uma separação modesta entre grupos, o que é coerente com uma exatidão de classificação de **68.44%**: as *features* acústicas contêm sinal discriminativo genuíno — muito acima do acaso ($1/20 = 5\%$) — mas a sobreposição natural entre gêneros impede uma separação nítida. Por outras palavras, o *clustering* encontra estrutura onde a classificação confirma que essa estrutura é informativa, mas não determinística.

A projeção PCA bidimensional das previsões da *Random Forest* (Figura 16)

, com os pontos coloridos segundo o gênero previsto, permite visualizar a sobreposição entre classes que explica simultaneamente a *silhouette* modesta e a exatidão inferior a 100%.

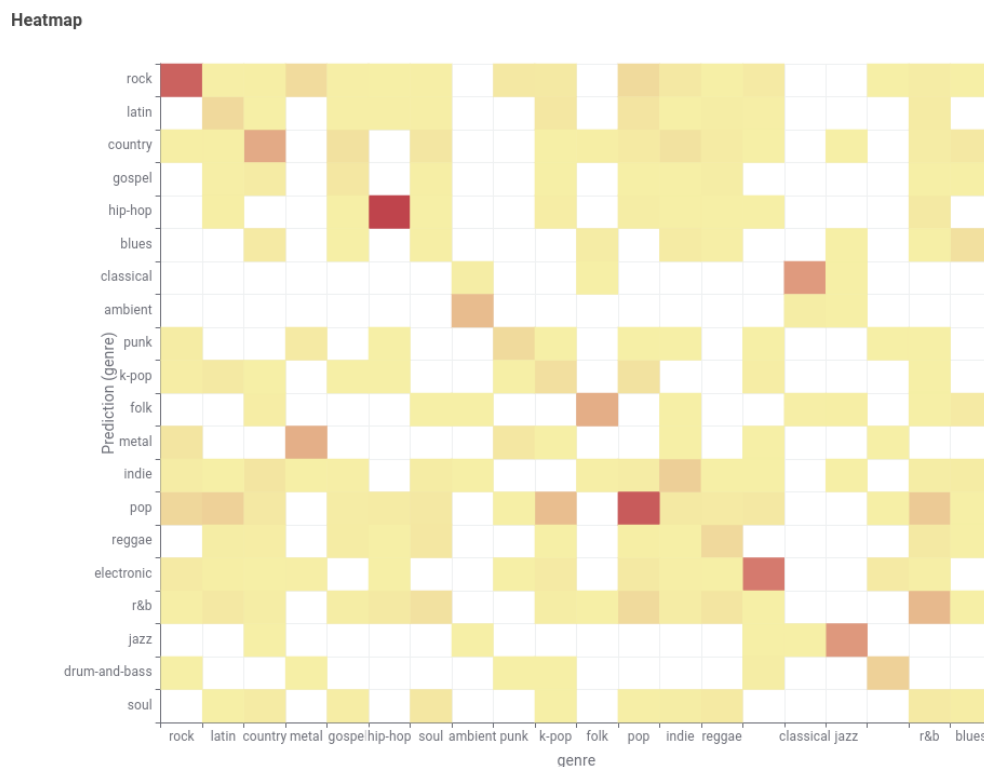


Figura 10: Matriz de confusão para o modelo Random Forest

3.4.3 Regressão da popularidade

Configuração. O ramo de regressão foi substancialmente expandido face à versão anterior. Além dos 13 preditores originais (`release_year`, `explicit`, as 10 *features* de áudio, `mode` e `duration_min`), o nó **One to Many** (#221) codifica os 20 géneros musicais em colunas binárias (*one-hot encoding*), totalizando **33 preditores**. Esta expansão é a concretização da hipótese de que o género musical transporta informação complementar às *features* acústicas para explicar a variabilidade da popularidade.

O nó **Expression** (#229) cria a variável auxiliar `popularity_class` (Baixa se `popularity` ≤ 29 , Média se ≤ 60 , Alta caso contrário) para estratificação da partição. A normalização Min-Max é aprendida no treino (79 482 linhas) e aplicada ao teste (19 871 linhas), sem fuga de informação. Foram treinados três modelos: regressão linear múltipla, árvore de regressão e *Random Forest*, todos com os 33 preditores e target `popularity`.

Regressão linear múltipla. O modelo linear, treinado com **Linear Regression Learner** (#163), inclui constante e utiliza todos os preditores disponíveis. As previsões foram avaliadas com **Numeric Scorer** (#165). O diagnóstico visual incluiu um gráfico de dispersão de valores reais *vs.* previstos e um histograma dos resíduos (Figuras 13 e 13)

Árvore de regressão. Para a árvore de regressão, foram avaliadas várias configurações experimentais exportadas, refletindo diferentes níveis de poda/regularização. Entre as configurações testadas, a melhor foi a experiência `5_40_20`, que obteve o melhor compromisso entre erro médio e estabilidade. Esta etapa é metodologicamente importante porque, ao contrário da versão exploratória inicial da árvore, demonstra que a poda altera de forma material a generalização do

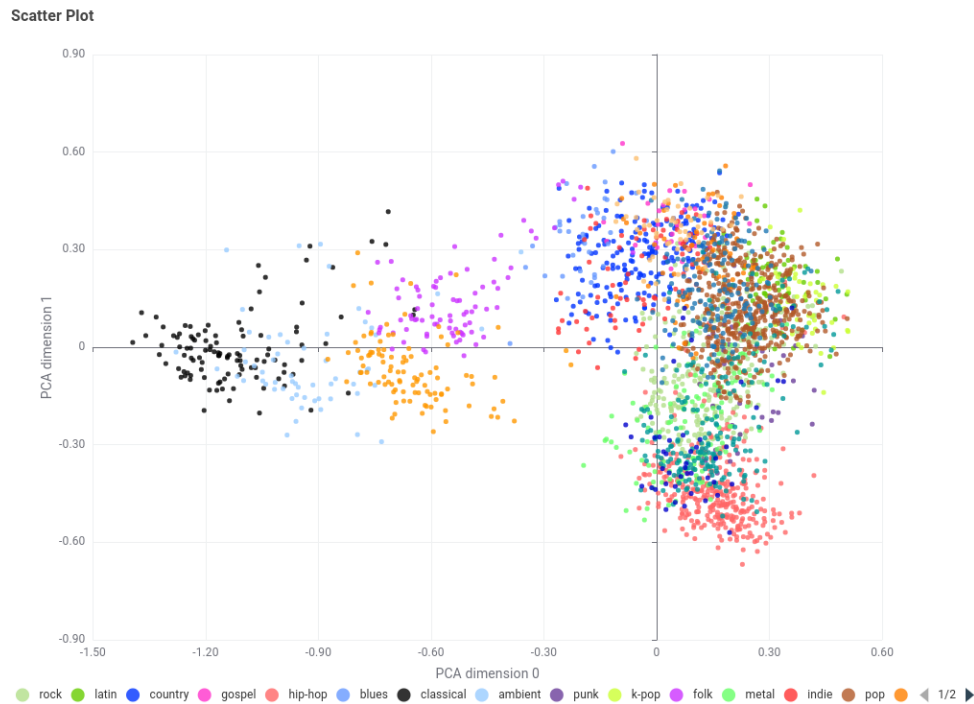


Figura 11: Projeção PCA das previsões da Random Forest

modelo. Os diagnósticos encontram-se nas Figuras 14 e 14

Random Forest. O modelo de *Random Forest*, configurado no nó **Random Forest Learner (Regression)** (#172), utiliza 100 árvores, *seed*=1, *bootstrap* ativo e amostragem de colunas em modo $\sqrt{\cdot}$ (*SquareRoot*). Os resultados exportados mostram que este modelo foi de facto executado e comparado com a regressão linear e com a árvore simples afinada. Os gráficos de diagnóstico encontram-se nas Figuras 15 e 15

3.5 Avaliação

A avaliação quantitativa da classificação baseou-se na exatidão, erro e coeficiente Kappa de Cohen, calculados sobre os conjuntos de teste através do nó **Scorer** (Tabela 2, apresentada na secção anterior). Para a regressão, a avaliação baseou-se em R^2 , MAE e RMSE, calculados via **Numeric Scorer**. Nesta versão do relatório continuam por integrar apenas os resultados exportados da MLP; os restantes resultados principais da Tarefa 1 já se encontram consolidados. A avaliação qualitativa é desenvolvida na secção de análise crítica (Secção 4).

Tabela 3: Métricas de regressão — Spotify Tracks (com género em *one-hot*)

Modelo	R^2	RMSE	MAE
Regressão linear	0.0459	18.194	13.718
Árvore de regressão (melhor configuração 5_40_20)	0.0346	18.301	13.808
<i>Random Forest</i>	0.0321	18.325	13.896

4 Resultados e análise crítica

4.1 Leitura crítica do *clustering*

Os resultados de *clustering* mostram que existe estrutura nos dados musicais, mas a separação entre grupos não é nítida. Um coeficiente de *silhouette* de 0.0883 no método hierárquico constitui um sinal de sobreposição relevante entre *clusters*, o que é consistente com a natureza do domínio: diferentes faixas podem partilhar níveis semelhantes de energia, valência e dançabilidade sem que isso se traduza em grupos radicalmente distintos. Em termos práticos, o *clustering* revela-se mais útil como ferramenta de segmentação exploratória do que como mecanismo de rotulagem rígida.

A análise dos centróides por *feature* permite identificar perfis sonoros diferenciados, e a composição dos *clusters* por género musical confirma que os agrupamentos capturam afinidades estilísticas genuínas, ainda que com sobreposição. Em particular, o *cluster_0* apresenta o perfil mais acústico e instrumental (*acousticness* = 0.708; *instrumentalness* = 0.751), funcionando como um polo de faixas mais contemplativas e menos energéticas; o *cluster_1* concentra as faixas mais energéticas e rápidas (*energy* = 0.843; *tempo* = 0.614); e o *cluster_2* distingue-se pelo nível muito elevado de *danceability* (0.806) e *speechiness* (0.581), aproximando-se de géneros de matriz urbana. Os valores médios de *silhouette* para *k-means* foram 0.1802 para $k = 4$, 0.1805 para $k = 5$ e 0.1619 para $k = 6$, o que mostra que $k = 4$ e $k = 5$ são praticamente equivalentes em qualidade geométrica. A preferência por $k = 5$ resulta, assim, menos de um ganho quantitativo e mais de uma melhor legibilidade semântica dos perfis obtidos.

4.2 Leitura crítica da classificação de género

Os resultados atualmente disponíveis para classificação demonstram que as 10 *features* acústicas contêm sinal discriminativo substancial para a previsão do género musical. A família *Random Forest* estabiliza em torno de 68% de exatidão e Kappa em torno de 0.65, com o melhor resultado exportado a atingir **68.44%** e **0.6571**. Este é um resultado expressivo quando comparado com a *baseline* aleatória de 5%, mas a própria estabilidade dos valores entre as melhores configurações também é informativa: o problema parece atingir rapidamente um teto de desempenho, sinal de que a limitação dominante está nos dados e não apenas no algoritmo.

O estudo de sensibilidade da *Random Forest* reforça esta leitura. As variantes menos expressivas apresentam degradação forte (por exemplo, 59.68% em *gini_6_5_100* e 63.50% em *gini_8_5_100*), mas a partir das configurações intermédias o desempenho converge para um patamar estreito: 68.16% (*gini_16_5_100*), 68.38% (*gini_20_1_100*), 68.36% (*gini_20_1_200*) e 68.44% (*gini_20_1_500*). A troca do critério Gini por *Information Gain Ratio* não melhora o desempenho em configurações comparáveis (68.38% vs. 68.20% nos modelos de 100 árvores), o que indica que a fronteira de decisão é relativamente robusta à variação do critério de divisão.

A leitura por classe mostra um comportamento muito desigual. A *Random Forest* distingue quase perfeitamente géneros com assinatura acústica mais estável, como *jazz* (*recall* = 0.9900), *classical* (0.9719), *hip-hop* (0.9496) e *ambient* (0.9487). Em contrapartida, classes como *soul* (0.2799), *k-pop* (0.4324), *latin* (0.4356), *r&b* (0.4442) e *gospel* (0.4539) revelam separação muito mais fraca. Esta assimetria é relevante: o modelo não falha de forma uniforme, mas sobretudo nos géneros mais híbridos ou estilisticamente próximos de outros grupos, sugerindo que a dificuldade do problema reside menos na complexidade do classificador e mais na sobreposição intrínseca entre classes no espaço acústico.

A projeção PCA das previsões (Figura 16)

confirma visualmente esta sobreposição: as diferentes classes formam manchas com contornos difusos, com algumas classes mais compactas e outras dispersas, refletindo a variabilidade intra-género das propriedades acústicas.

4.3 Relação entre *clustering* e classificação

A coexistência dos dois ramos sobre o mesmo espaço de *features* permite uma triangulação metodológica rara, com conclusões convergentes:

1. **Ambas as abordagens encontram sinal.** O *k-means* identifica 5 grupos acústicos não aleatórios (*silhouette* positivo e estável entre 0.1619 e 0.1805). A classificação supervisionada recupera o género com 68.44% de exatidão (muito acima dos 5% do acaso). Isto demonstra que as *features* acústicas do Spotify têm, de facto, poder discriminativo.
2. **A separação não é total.** A *silhouette* modesta (0.0883 no método hierárquico; 0.1619–0.1805 no *k-means*) é a contrapartida não supervisionada da exatidão de classificação de 68.44% — ambas apontam para uma sobreposição intrínseca no espaço acústico que nenhum modelo, por mais flexível que seja, consegue eliminar.
3. **O *clustering* descobre agrupamentos que a classificação valida.** A matriz *cluster* $\times$ *género* (Figura 17) mostra que géneros próximos no espaço acústico coabitam os mesmos *clusters*, e a matriz de confusão da classificação revela que as trocas entre géneros seguem precisamente este mesmo padrão de afinidade acústica. Esta convergência entre uma técnica não supervisionada e outra supervisionada reforça a validade de ambas.
4. **Limite fundamental da representação acústica.** O facto de o melhor classificador atualmente documentado não ultrapassar 68.44% de exatidão estabelece um limite superior empírico para o que é possível extrair das 10 *features* de áudio. Este limite explica por que razão o *k-means* produz *clusters* com *silhouette* modesta: a própria representação dos dados impõe um teto à separabilidade.

4.4 Leitura crítica da regressão

A regressão da popularidade confirma a natureza estruturalmente difícil do target. Mesmo após a inclusão do género por *one-hot encoding*, todos os modelos permanecem com R^2 muito baixos: a regressão linear é a melhor das três abordagens com $R^2 = 0.0459$, seguida pela melhor árvore afinada com $R^2 = 0.0346$, e pela *Random Forest* com $R^2 = 0.0321$. Em termos absolutos, os três modelos operam num intervalo muito estreito de erro (RMSE entre 18.19 e 18.33; MAE entre 13.72 e 13.90), o que mostra que aumentar a flexibilidade do modelo não altera materialmente a qualidade preditiva.

Este resultado é analiticamente importante por duas razões. Primeiro, a regressão linear superar os modelos não lineares sugere que o sinal disponível é simultaneamente fraco e relativamente simples; se existisse estrutura não linear rica entre as *features* e a popularidade, seria expectável que a árvore afinada ou a *Random Forest* explorassem essa vantagem. Segundo, a proximidade entre a melhor árvore simples e a *Random Forest* indica que, uma vez controlado o

sobreajustamento por poda/regularização, a componente não linear do problema continua a ser reduzida.

A própria trajetória da árvore de regressão é instrutiva. As configurações testadas variam pouco entre si após a poda, e a melhor experiência (5_40_20) melhora apenas marginalmente as restantes. Isto sugere que a afinação resolve o problema do sobreajustamento grosseiro da árvore exploratória inicial, mas não cria capacidade preditiva nova: o limite continua a ser o conteúdo informativo do dataset e não o hiperparâmetro escolhido.

Globalmente, a regressão é a vertente onde o contraste entre *clustering*/classificação e popularidade se torna mais visível. As mesmas *features* acústicas que conseguem segmentar perfis sonoros e classificar géneros com qualidade razoável quase não conseguem prever a popularidade. Isto implica que a popularidade, embora marginalmente afetada pelo género, depende sobretudo de fatores externos ao espaço acústico: capital simbólico do artista, marketing, exposição em playlists, momento cultural e dinâmica social de consumo. Em suma, o dataset é adequado para estudar identidade sonora, mas apenas parcialmente adequado para estudar sucesso comercial.

4.5 Limitações

A principal limitação é a natureza sintética do *dataset* que, embora gerado com distribuições inspiradas em dados reais, não substitui plenamente um catálogo musical autêntico. As relações entre *features* acústicas e popularidade podem ser artificialmente atenuadas ou amplificadas pelo processo de geração, e a distribuição dos géneros (uniforme, 20 géneros com igual representação) não reflete a realidade do mercado musical, onde géneros como *pop* e *hip-hop* dominam em volume de produção.

Adicionalmente, os valores de *silhouette* obtidos, embora coerentes entre os métodos hierárquico e *k-means*, variam com a amostragem (0.0883 em 5 000 registos vs. 0.1619–0.1805 em 20 000), o que sugere que a estimativa da qualidade do agrupamento é sensível ao tamanho e à composição da amostra. As futuras métricas de regressão poderão ainda ser influenciadas pelas estratégias de imputação e de tratamento de *outliers* adotadas na preparação.

Por fim, a arquitetura de três ramos paralelos (*clustering*, classificação, regressão) sobre o mesmo *dataset* introduz o risco de *data leakage* conceptual se as conclusões de um ramo forem usadas para justificar decisões noutra ramo sem o devido cuidado metodológico. A separação estrita dos fluxos no KNIME mitiga este risco operacionalmente, mas a independência analítica deve ser mantida na interpretação.

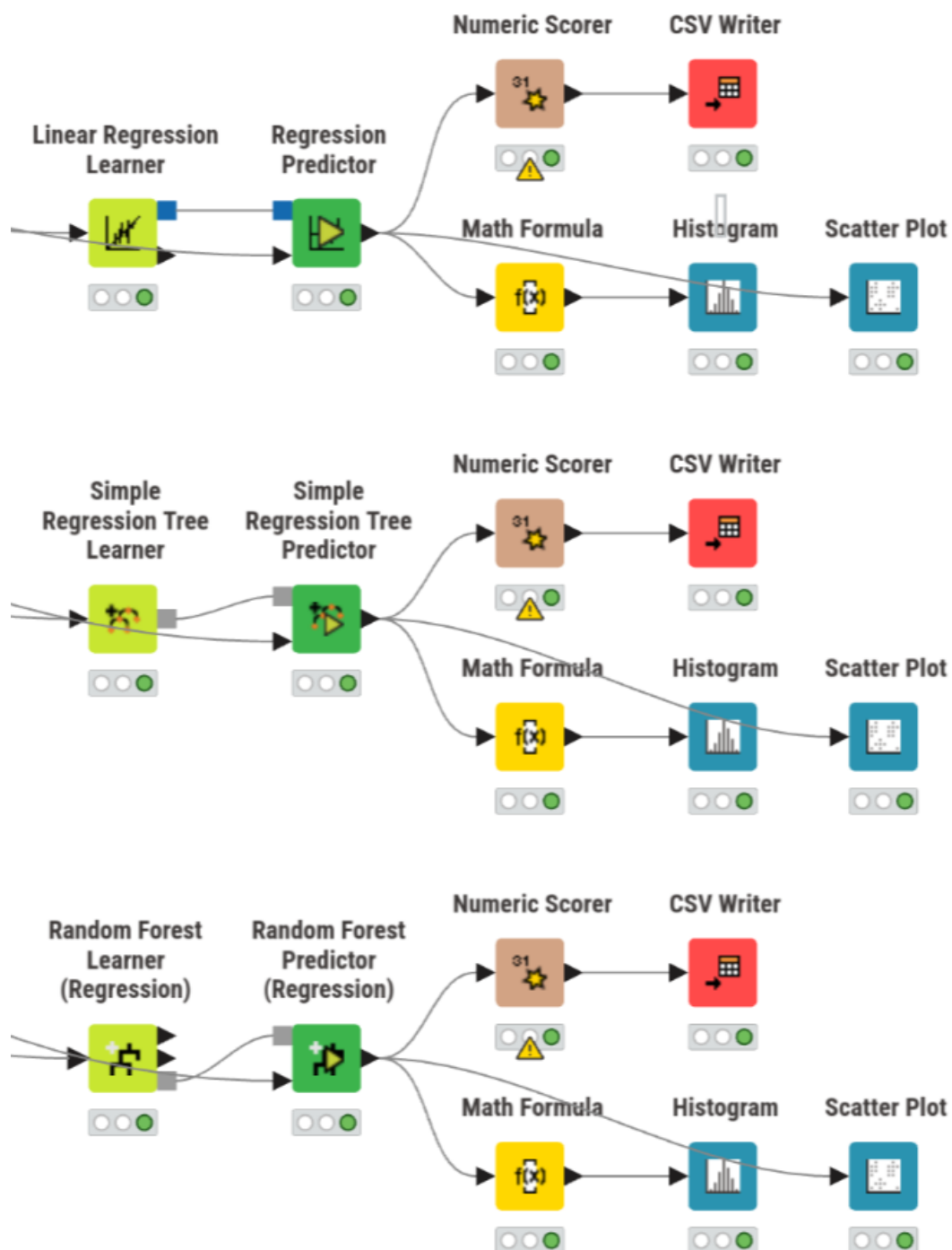


Figura 12: Workflow da fase de Regressão

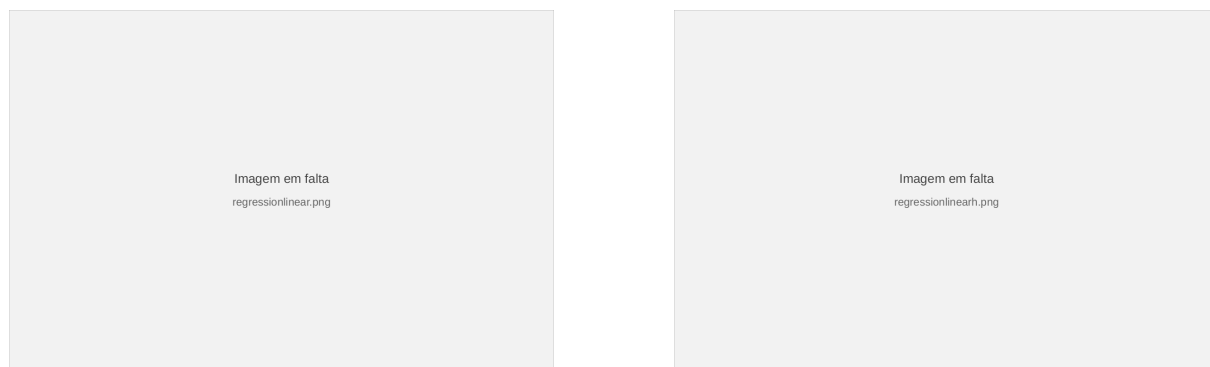


Figura 13: Diagnóstico do modelo linear: dispersão vs previsão e histograma de resíduos

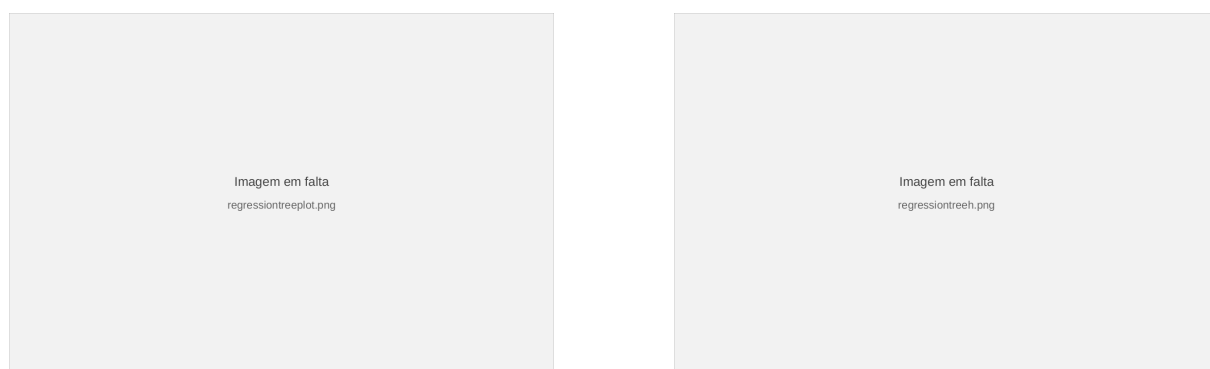


Figura 14: Diagnóstico da árvore de regressão

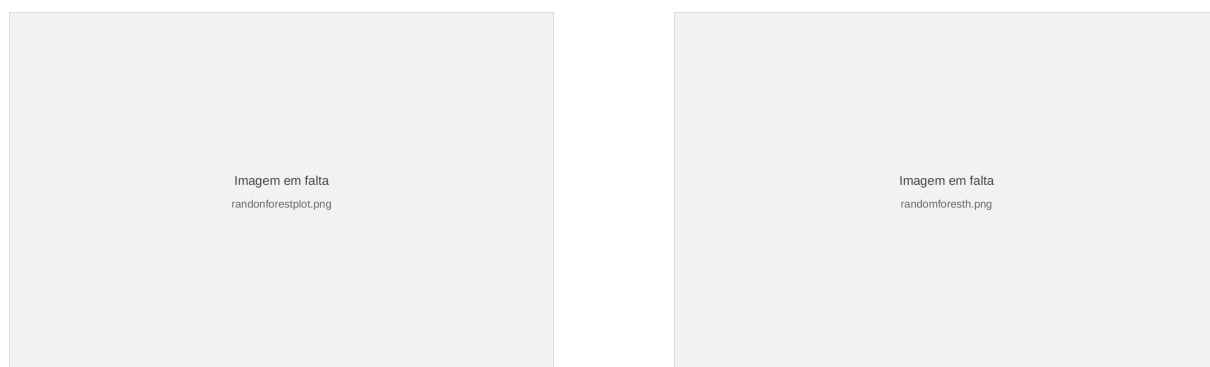


Figura 15: Diagnóstico da Random Forest: dispersão vs previsão e histograma de resíduos

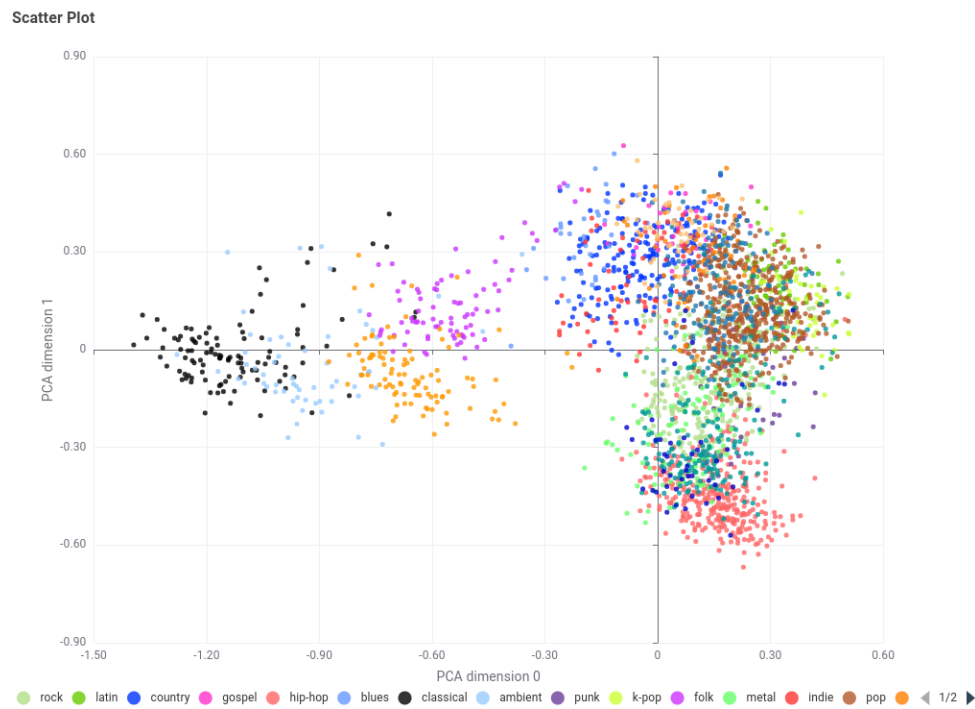


Figura 16: Projeção PCA das previsões da Random Forest

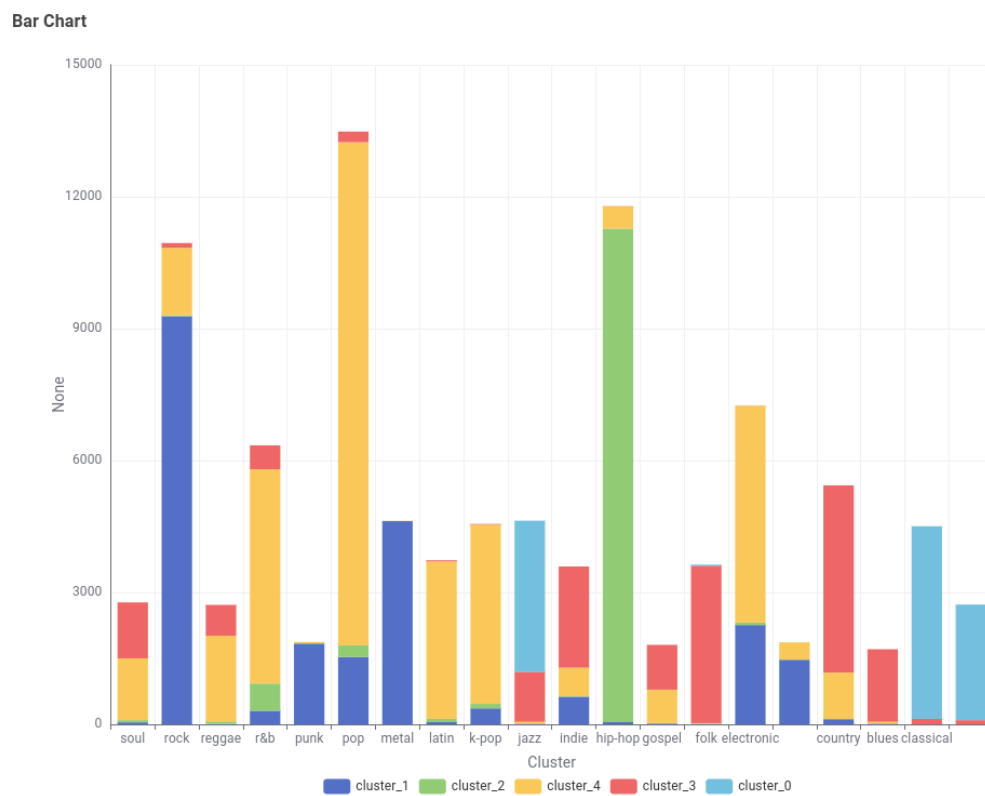


Figura 17: Distribuição dos géneros musicais por cluster

Parte II — Tarefa 2: Dataset Atribuído (Consumo Energético)

5 Definição da metodologia

A metodologia de análise de dados que será utilizada neste problema é a SEMMA: Sample, Explore, Modify, Model e Assess. A fase de Sample encontra-se já concretizada na forma do dataset atribuído, pelo que o presente relatório se foca nas fases seguintes. Assim, começou-se por realizar a fase de Explore ou Exploração, cujo objetivo é perceber potenciais tendências e problemas com os dados, para na fase de Modificação se fazerem transformações que visam corrigir os problemas identificados na fase de Exploração. Por fim, aplicam-se as fases de Modelação e Avaliação para cada um dos 5 modelos desenvolvidos para este problema.

5.1 Visão geral do workflow desenvolvido no KNIME

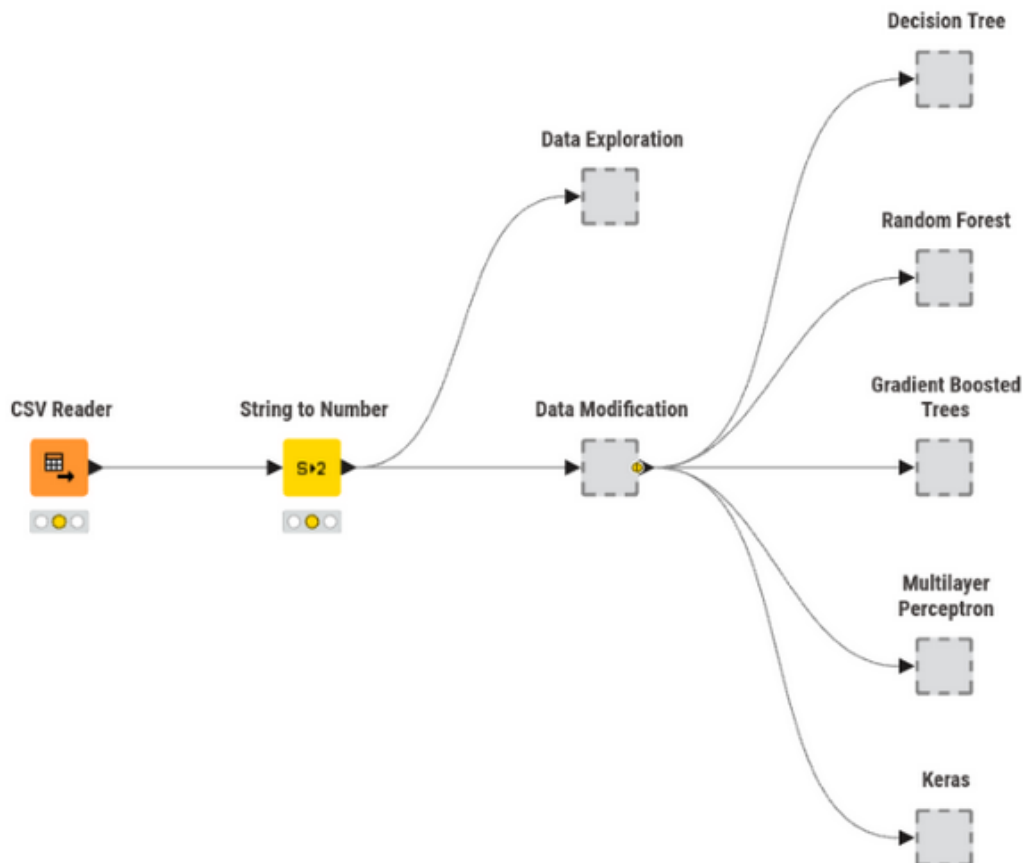


Figura 18: Visão geral do workflow desenvolvido no KNIME

De forma geral, o workflow está organizado num conjunto de metanodes que encapsulam os diferentes passos. O nó String to Number, que converte features numéricas que aparecem no dataset em formato string, encontra-se fora de qualquer metanode de forma a propagar os missing values nessas features para a Exploração e para a Modificação simultaneamente. No

final, cada um dos 5 modelos recebe o output das transformações dos dados, sendo a Avaliação feita dentro de cada metanode.

5.2 Exploração

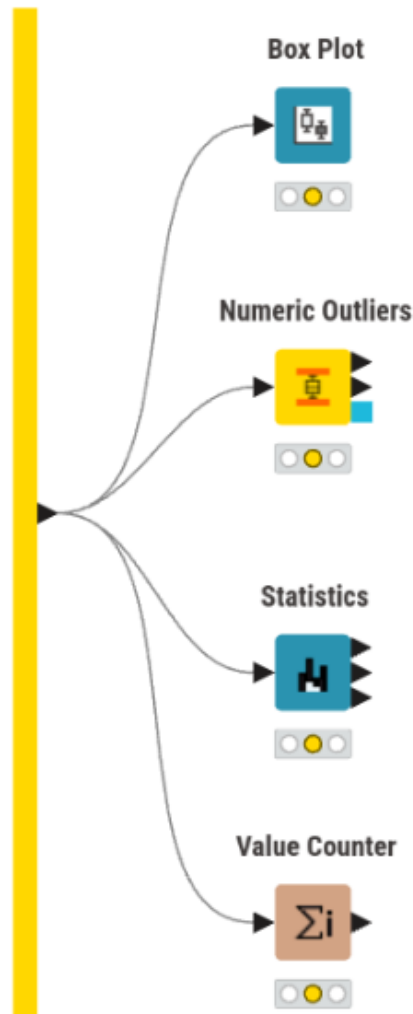


Figura 19: Exploração

O primeiro passo da exploração dos dados foi a utilização do nó Statistics para verificar diversas estatísticas, destacando-se as seguintes observações:

- Precipitation e rain apresentam muitos missing values ($\sim 90\%$)
- Snowfall tem todos os valores a 0 (mean = min = max = 0)
- RowID é redundante
- Todas as restantes features apresentam uma quantidade semelhante de missing values, cerca de 10%. Por isso, deixa de ser viável remover as entradas com missing values, uma vez que, feito isso, o dataset final fica com 1300 entradas (apenas 15% do total). Portanto, foram avaliadas, para cada modelo, diferentes abordagens para imputar os missing values (média e mediana para os numéricos e moda ou valor fixo “Missing” para as strings).

De seguida, foi verificada a presença de outliers, recorrendo tanto a Numeric Outliers para obter resultados numéricos como ao Box Plot para fazer uma exploração mais visual. Desta análise, retirou-se que várias features (como as do vento e as da voltagem) apresentam outliers em quantidade reduzida ($<2,5\%$) e a `surface_pressure` ligeiramente mais ($\sim 5\%$). No total, são 1700 as entradas com algum outlier, pelo que a sua remoção não foi considerada, pois perder-se-ia uma boa parte do dataset (cerca de 20%). Portanto, na avaliação dos modelos, os outliers foram ou mantidos ou ajustados para o valor permitido mais próximo, de maneira a avaliar qual o melhor tratamento para cada modelo.

Por fim, foi ainda feita uma análise dos valores das features categóricas com Value Counter, para determinar se existem valores fora do padrão. Concluiu-se que as features `Diffuse` e `Direct Radiation` não apresentam valores anormais. Já a `target`, `Consumptions`, apresenta, em algumas entradas, valores com erros ortográficos (e.g., `Hgh` e `Lw`) ou escritos em português em vez de inglês (e.g., `MedioAlto` vs `Medium-High Consumption`).

5.3 Transformação

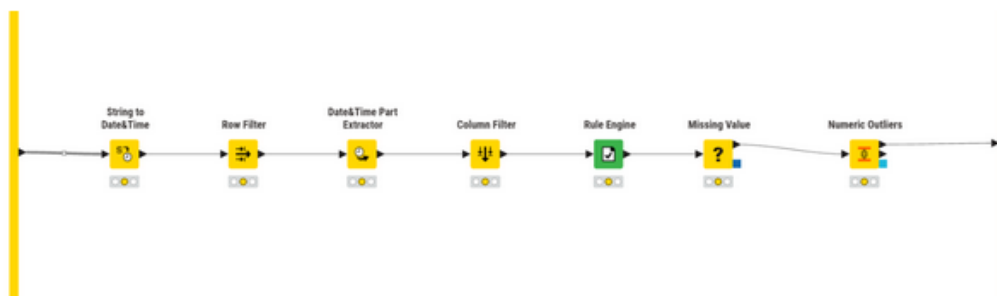


Figura 20: Transformação

As primeiras transformações aplicadas aos dados centraram-se nos formatos das features, incluindo a conversão das colunas numéricas que se encontravam em string (`Medium Voltage` e `Total`), que foi realizada fora deste metanode, conforme descrito anteriormente, e a passagem das datas de string para um formato adequado, além da consequente extração dos campos mais relevantes para este problema (hora, dia da semana e mês). A feature original com a data completa foi removida para evitar introduzir ruído. Ainda, foram detectadas 4 linhas nas quais a data era inválida (e.g., `2025-11-32`), pelo que essas entradas foram removidas, por ser uma quantidade muito reduzida.

De seguida, no seguimento da exploração realizada, foram removidas algumas features, sendo elas:

- `RowID` (redundante)
- `precipitation` e `rain` ($\sim 90\%$ missing values)
- `snowfall` (redundante, $\text{mean} = \text{min} = \text{max} = 0$)

Por fim, de acordo com o que foi possível perceber na exploração, foi aplicada uma Rule Engine para uniformizar o formato dos valores de `Consumptions`, resultando daqui 5 valores possíveis (`Low`, `MediumLow`, `Medium`, `MediumHigh`, `High`).

6 Modelação e Avaliação

É agora altura de desenvolver modelos para o presente problema. Foram concebidos 5 modelos de classificação, com 3 baseados em árvores e 2 redes neuronais. Algumas observações relevantes para as secções seguintes:

1. A partição dos dados para treino e teste foi feita, sempre que possível, com recurso aos nós X-Partitioner e X-Aggregator, uma vez que o uso de cross-validation, embora mais custoso a nível computacional, permite aproveitar a totalidade do dataset e reduzir as chances do modelo sofrer de overfitting;
2. Os nodos de cross-validation foram configurados com uma random seed, de modo a ser possível obter resultados reproduzíveis e sem variabilidade aleatória;
3. Não foram testadas todas as combinações possíveis de transformações de features e configurações dos modelos, pois é uma quantidade impraticável. Por isso, a abordagem adotada foi: testar um conjunto de abordagens e as suas combinações; tirar daí o melhor resultado; realizar o próximo conjunto de testes com a combinação obtida anteriormente. Isto pode fazer com que não sejam detectadas as melhores combinações para cada modelo, mas torna este processo de refinamento mais exequível.

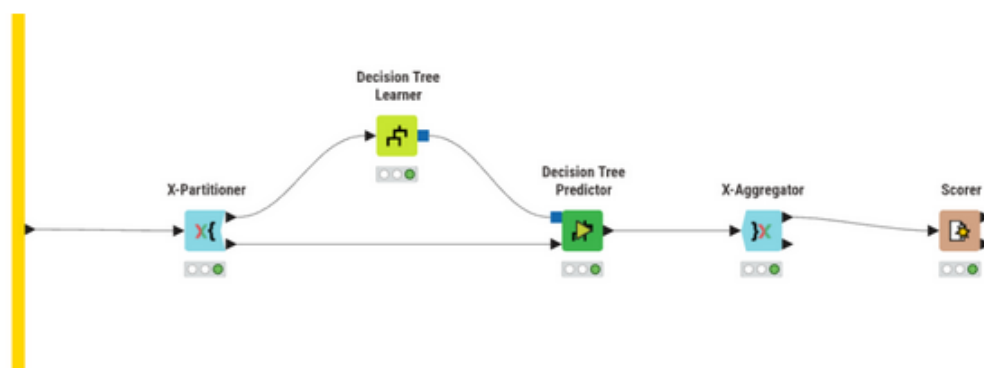


Figura 21: Decision Tree

Tal como mencionado na fase de exploração, os tratamentos de missing values testados foram média e mediana para os valores numéricos e, para os categóricos, moda e valor fixo “Missing”. Já os outliers, tal como dito anteriormente, foram mantidos ou ajustados para o valor permitido mais próximo.

Os resultados obtidos mostram que o tratamento de missing values e outliers teve impacto reduzido no desempenho da Decision Tree, verificando-se diferenças muito pequenas entre as diferentes estratégias avaliadas (menos de 1%). Em contrapartida, os hiperparâmetros estruturais da árvore tiveram mais influência no desempenho final. Em particular, o aumento do número de records por nó levou a melhorias consistentes da accuracy e do Cohen’s Kappa. Observou-se também que a utilização de pruning não trouxe vantagens relevantes relativamente à ausência de pruning, obtendo resultados ligeiramente inferiores na maioria das configurações, mas reduzindo a variabilidade ao ajustar os outros parâmetros. Quanto à métrica de divisão utilizada, tanto o

Outliers	Missing values (números)	Missing values (categóricos)	Accuracy	Cohen's Kappa
manter	média	moda	0.884	0.852
manter	média	fixo	0.887	0.855
manter	mediana	moda	0.885	0.853
manter	mediana	fixo	0.888	0.857
ajustar	média	moda	0.885	0.852
ajustar	média	fixo	0.89	0.859
ajustar	mediana	moda	0.885	0.853
ajustar	mediana	fixo	0.889	0.858

Quality Measure	Pruning Method	Min num recs per node	Accuracy	Cohen's Kappa
Gini index	No pruning	2	0.89	0.859
Gini index	No pruning	4	0.897	0.869
Gini index	No pruning	8	0.919	0.897
Gini index	No pruning	12	0.927	0.906
Gini index	MDL	2	0.925	0.904
Gini index	MDL	4	0.925	0.904
Gini index	MDL	8	0.925	0.904
Gini index	MDL	12	0.926	0.905
Gain ratio	No pruning	2	0.888	0.857
Gain ratio	No pruning	4	0.904	0.878
Gain ratio	No pruning	8	0.918	0.895
Gain ratio	No pruning	12	0.922	0.901
Gain ratio	MDL	2	0.919	0.897
Gain ratio	MDL	4	0.919	0.897
Gain ratio	MDL	8	0.921	0.899
Gain ratio	MDL	12	0.921	0.899

Gini Index como o Gain Ratio produziram desempenhos semelhantes, embora o Gini Index tenha apresentado, de forma consistente, resultados ligeiramente superiores. O melhor desempenho global foi obtido com Gini Index, sem pruning e com mínimo de 12 registos por nó, atingindo uma accuracy de 0.927 e um Cohen's Kappa de 0.906.

6.1 Random Forest

Aqui foram testadas as mesmas combinações de tratamento de missing values e outliers que na Decision Tree.

(Com Number of models = 100)

Tal como observado na Decision Tree, as diferentes estratégias de tratamento de missing values e outliers tiveram impacto praticamente nulo no desempenho da Random Forest. Relativamente aos hiperparâmetros da Random Forest, verificou-se que limitar o número de níveis das árvores provocou uma redução consistente do desempenho. Isto sugere que este problema beneficia de árvores mais profundas, capazes de capturar relações mais complexas entre as features. Também o aumento do parâmetro Minimum child node size levou a pequenas perdas de desempenho. Os diferentes critérios de divisão testados (Information Gain Ratio, Information

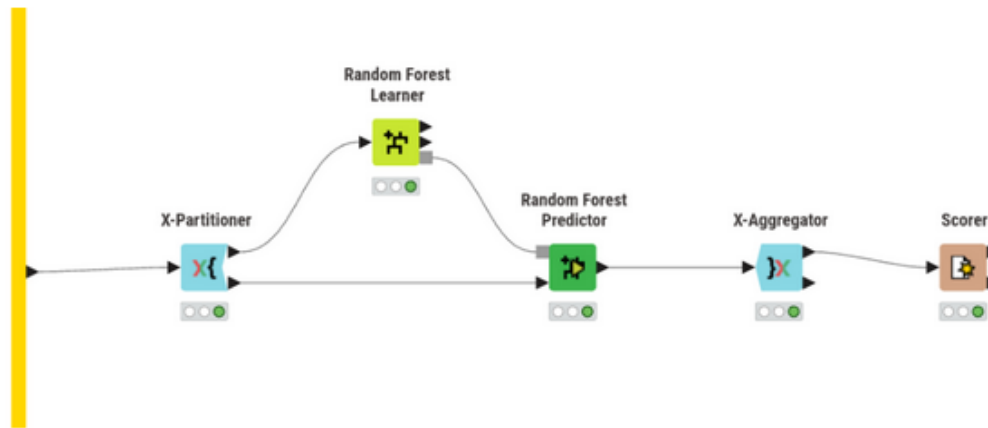


Figura 22: Random Forest

Outliers	Missing values (números)	Missing values (categóricos)	Accuracy	Cohen's Kappa
manter	média	moda	0.929	0.909
manter	média	fixo	0.929	0.909
manter	mediana	moda	0.93	0.91
manter	mediana	fixo	0.929	0.909
ajustar	média	moda	0.929	0.909
ajustar	média	fixo	0.928	0.908
ajustar	mediana	moda	0.929	0.909
ajustar	mediana	fixo	0.928	0.908

Gain e Gini) apresentaram desempenhos muito semelhantes, embora o Information Gain Ratio tenha obtido, de forma consistente, os melhores resultados. Finalmente, a avaliação do número de modelos mostrou que aumentar o número de árvores acima de 100 não produziu melhorias relevantes. Isto indica que a Random Forest atinge rapidamente estabilidade neste problema, sendo que ensembles maiores apenas aumentam o custo computacional sem ganhos significativos de desempenho.

6.2 Gradient Boosted Trees

Ao contrário do que foi feito até agora, não foram avaliadas as estratégias de imputação de missing values, uma vez que este modelo dispõe de mecanismos para lidar com este problema. Sendo assim, foram testados estes métodos juntamente com o tratamento de outliers com as configurações padrão, antes de começar a variar os hiperparâmetros.

É possível verificar que a configuração com ajuste de outliers tem um resultado igual a não tratar outliers (usando em ambos os casos XGBoost), pelo que se optou por utilizar esta última abordagem, por não introduzir dados artificiais. Na avaliação que se segue, a variação do learning rate foi feita em conjunto com o número de modelos, ou seja, diminuindo a learning rate, aumenta-se o número de modelos. Isto foi feito porque, ao diminuir a learning rate, cada árvore individual contribui menos para o resultado final, pelo que é necessário aumentar o total de árvores para compensar.

Os resultados mostram que diferentes combinações de learning rate e número de modelos

Split criterion	Limit number of levels	Minimum child node size	Accuracy	Cohen's Kappa
Information Gain Ratio	off	off	0.93	0.91
Information Gain Ratio	off	2	0.929	0.909
Information Gain Ratio	off	4	0.927	0.907
Information Gain Ratio	10	off	0.919	0.896
Information Gain Ratio	10	2	0.919	0.896
Information Gain Ratio	10	4	0.918	0.895
Information Gain	off	off	0.929	0.909
Information Gain	off	2	0.928	0.908
Information Gain	off	4	0.927	0.907
Information Gain	10	off	0.922	0.9
Information Gain	10	2	0.921	0.899
Information Gain	10	4	0.921	0.898
Gini	off	off	0.929	0.909
Gini	off	2	0.928	0.907
Gini	off	4	0.927	0.907
Gini	10	off	0.92	0.898
Gini	10	2	0.92	0.897
Gini	10	4	0.921	0.899

Number of models	Accuracy	Cohen's Kappa
50	0.929	0.909
100	0.93	0.91
150	0.93	0.91
200	0.929	0.909
250	0.929	0.909

conduzem praticamente ao mesmo desempenho máximo (accuracy = 0.932 e Cohen's kappa = 0.913), sugerindo que, para este problema, o aumento do número de modelos juntamente com a redução da learning rate não produzem ganhos relevantes. Em contrapartida, observou-se que configurações com maior profundidade das árvores e com row sampling ativo tendem a apresentar desempenhos ligeiramente superiores, indicando que estes parâmetros têm maior influência no resultado final do que a combinação learning rate / número de modelos.

6.3 Multilayer Perceptron

Para as redes neurais, uma transformação de dados importante é a normalização (e consequente desnormalização), pelo que foram testados 3 métodos diferentes para a normalização dos dados. No caso do método Min-max, usar o intervalo

$$0; 1$$

produziu resultados bastante medíocres. Por isso, o intervalo usado foi

$$-1; 1$$

, que melhorou a performance da rede por centrar os dados em zero. Ainda, o método decimal scaling também produziu resultados pouco satisfatórios (accuracy na ordem dos 60%), pelo que

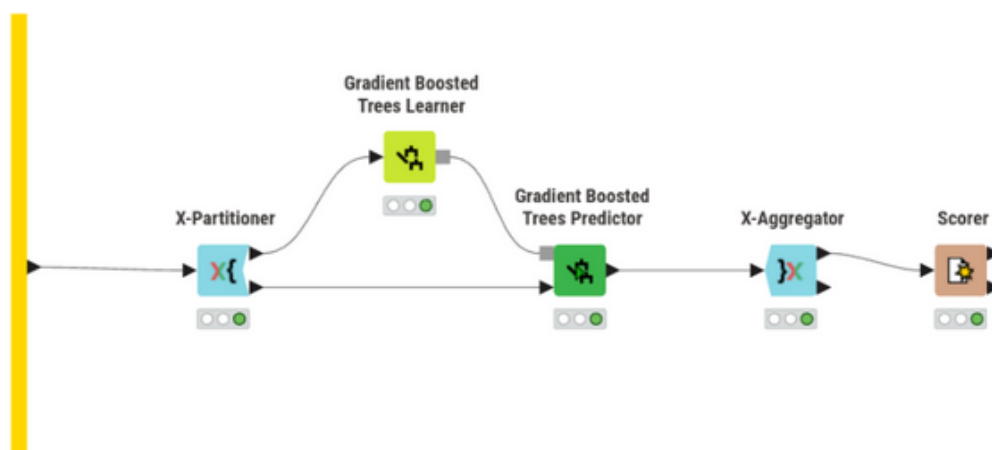


Figura 23: Gradient Boosted Trees

Outliers	Missing values	Accuracy	Cohen's Kappa
manter	XGBoost	0.928	0.908
manter	Surrogate	0.897	0.868
ajustar	XGBoost	0.928	0.908
ajustar	Surrogate	0.896	0.867

não foi avaliado com o mesmo detalhe que foram os outros. A outra transformação essencial tem a ver com as features categóricas nominais, que não são adequadas para dar como input a uma rede neuronal. Foram testadas duas formas de as tornar apropriadas ao modelo:

- **numerar:** atribuir valores numéricos sequenciais às categorias. Esta forma de tratamento pode não ser 100% adequada porque introduz na feature uma noção de ordem que pode não ser verdadeira/aplicável. No entanto, para este problema, as features afetadas usam classificações como None, Low, Medium e High, pelo que a numeração pode fazer sentido;
- **One to Many:** nó que, para cada categoria de uma feature, cria uma nova feature cujo nome é o nome da original junto com cada valor, e atribui 0 ou 1 conforme o valor da feature naquela entrada. Este método permite tornar features categóricas adequadas para redes neurais, sem introduzir noções de ordem como a simples numeração.

Um detalhe importante, que difere consoante qual das duas abordagens é utilizada, é quando se faz a normalização. No caso da numeração, a normalização é feita depois, de modo a englobar os novos valores criados. Assim, evita-se que a feature fique com uma escala diferente das restantes. Já no caso de se usar o One to Many, a normalização é feita antes, pois os valores já estão numa escala consistente (0 ou 1) e assim não se destrói a interpretação binária, que é precisamente o objetivo desta abordagem.

Considerando tudo isto, começou-se por avaliar todas as combinações destas configurações, uma vez que são dependentes umas das outras. Por exemplo, os melhores tratamentos de missing values e outliers com Z-score podem ser os piores para Min-max, e portanto a avaliação não estaria correta. O único método de normalização não avaliado com a mesma profundidade que os outros dois foi o Decimal Scaling, porque fez o modelo obter uma accuracy de ~60%

Learning Rate	Number of Models	Limit number of levels	Row Sampling	Accuracy	Cohen's Kappa
0.1	100	4	off	0.928	0.908
0.1	100	4	0.5	0.929	0.909
0.1	100	4	0.8	0.929	0.909
0.1	100	8	off	0.929	0.909
0.1	100	8	0.5	0.931	0.912
0.1	100	8	0.8	0.932	0.913
0.05	200	4	off	0.928	0.908
0.05	200	4	0.5	0.929	0.909
0.05	200	4	0.8	0.929	0.909
0.05	200	8	off	0.928	0.908
0.05	200	8	0.5	0.932	0.913
0.05	200	8	0.8	0.932	0.912
0.02	400	4	off	0.928	0.908
0.02	400	4	0.5	0.928	0.908
0.02	400	4	0.8	0.928	0.908
0.02	400	8	off	0.93	0.91
0.02	400	8	0.5	0.932	0.913
0.02	400	8	0.8	0.932	0.913

com algumas combinações variadas dos restantes parâmetros em avaliação. Assim, foi possível reduzir ligeiramente o espaço de procura (48 para 32 combinações).

Estas combinações foram testadas com hiperparâmetros default do modelo (100 iterações, 1 hidden layer e 10 neurónios por layer).

Os resultados obtidos mostram que o desempenho do Multilayer Perceptron depende mais das transformações aplicadas aos dados do que os modelos anteriores, com ênfase para a normalização e a representação das variáveis categóricas. Dentre os métodos testados, a normalização Z-score apresentou consistentemente melhores resultados do que Min-max e Decimal Scaling. Relativamente às variáveis categóricas, a abordagem One to Many revelou-se superior à simples numeração das categorias neste problema.

Quanto à arquitetura da rede, verificou-se que aumentar o número de iterações melhorou consistentemente os resultados, indicando que o modelo beneficiou de mais tempo de treino. No entanto, o aumento do número de hidden layers e neurónios nem sempre conduziu a melhorias. Neste problema, os resultados sugerem que arquiteturas relativamente simples já conseguem capturar os padrões mais relevantes presentes no dataset. Assim, o aumento da profundidade e do número de neurónios introduziu maior complexidade de otimização sem acrescentar capacidade útil de generalização em relação aos dados de treino, levando em vários casos a uma degradação do desempenho no conjunto de teste.

6.4 Keras

O último modelo avaliado foi mais uma rede neuronal, desta vez usando a biblioteca Keras, que possibilita explorar arquiteturas mais flexíveis e diferentes métodos de otimização. Portanto, as transformações dos dados avaliadas para o MLP não foram reavaliadas, tendo-se optado por manter a melhor combinação encontrada (normalização Z-score, One to Many para as features

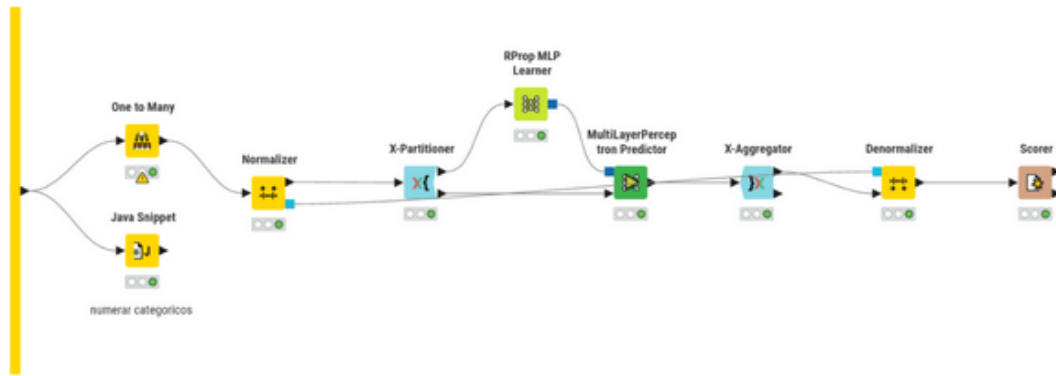


Figura 24: Multilayer Perceptron

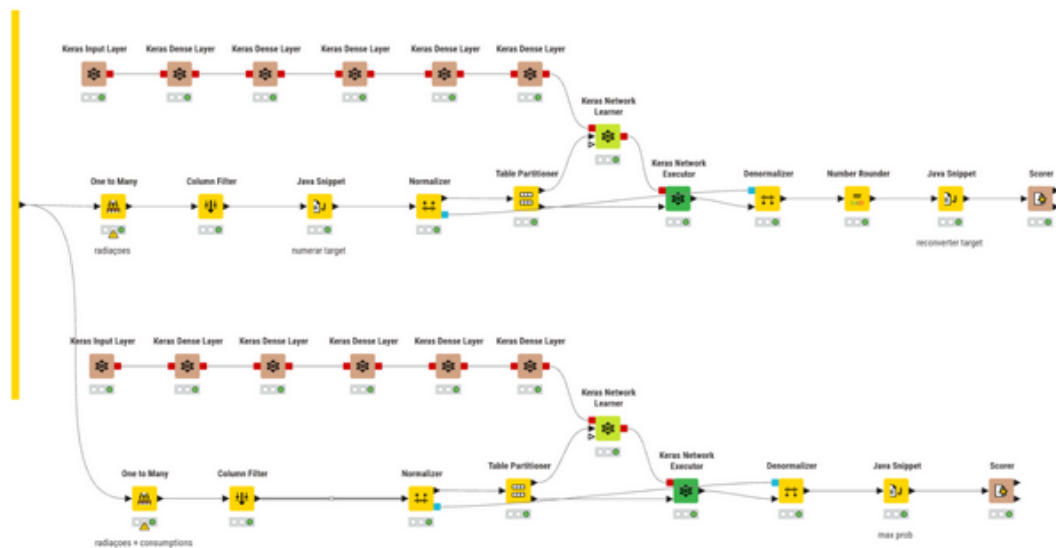


Figura 25: Keras

categóricas, ajuste de outliers, mediana para os missing values numéricos e moda para os missing values categóricos). Portanto, agora o foco passa a ser a definição da arquitetura da rede e do processo de treino. Antes de prosseguir para a arquitetura da rede, é necessário efetuar algumas transformações nos dados para ser possível aplicar a rede ao problema de classificação. Em particular, é preciso transformar a target Consumptions, pois ao contrário do RProp MLP, estas redes neuronais só recebem e devolvem valores numéricos como resultado. Desta forma, à semelhança do que aconteceu com o modelo anterior, foram testadas duas formas de fazer esta transformação:

- **One to Many:** cria-se uma coluna para cada possível valor da feature original, ou seja, cria-se um conjunto de booleanos. Para o output da rede, usam-se 5 neurónios, obtendo-se 5 valores de 0 a 1 que somam 1, ou seja, probabilidades. Depois é só escolher o maior para se fazer a previsão daquela entrada. Embora este formato de 0s ou 1s já seja aceite pela rede, a noção lógica destes valores é perdida, pelo que esta poderá não ser a melhor abordagem;

- **numerar:** numerar de 1 a 5 os possíveis valores de Consumptions. Este tratamento é adequado porque os valores da target já têm uma ordem própria ($\text{Low} < \text{MediumLow} < \text{Medium} < \text{MediumHigh} < \text{High}$), pelo que o significado da numeração continua a fazer sentido para a rede, ao contrário dos booleanos criados com o método anterior. Aqui, a camada de output passa a ter apenas 1 neurónio. O valor gerado pela rede é limitado ao intervalo válido e depois arredondado (e.g., $6.1 \Rightarrow 5$ e $0.2 \Rightarrow 1$), sendo por fim feita a conversão inversa para o valor em string correspondente.

Como configuração inicial, usou-se uma hidden layer com 16 neurónios e função de ativação ReLU (tal como em todas as futuras hidden layers adicionadas), além da camada de output com 1 ou 5 neurónios, conforme a transformação aplicada à target. Quanto às configurações desta última, no caso do One to Many, a loss function usada é a Categorical Cross Entropy, juntamente com a função de ativação Softmax, que transforma o output nas probabilidades mencionadas anteriormente. Já no caso de se utilizar a numeração, a loss function usada é a Mean Absolute Error e a função de ativação é a Linear, pois não se devem fazer modificações ao output da rede, de maneira a ser possível arredondar o valor produzido. Por fim, o otimizador utilizado em ambas as situações foi o Adam, com os parâmetros padrão. Depois da configuração inicial, foram testadas outras arquiteturas com mais hidden layers e também neurónios, mantendo o “afunilamento”, ou seja, reduzindo o número de neurónios em cada layer até chegar ao output. Foram utilizados números de neurónios múltiplos de 2 por conveniência experimental e alinhamento com práticas comuns de implementação. Após ser determinada a melhor arquitetura da rede e método de conversão da target, foram experimentados diferentes números de epochs, valor que se fixou inicialmente em 5 como compromisso entre capacidade de aprendizagem e custo computacional.

Nota: o uso de cross-validation tornou-se computacionalmente inviável com este modelo, cuja duração do treino se começa a tornar excessivamente longa numa máquina casual, especialmente com mais hidden layers e neurónios. Por isso, foi usado um Table Partitioner, fazendo uma partição de 70/30 com uma random seed fixa.

One to Many:

Numerar:

Os resultados obtidos mostram que a estratégia One to Many é significativamente mais adequada para este problema de classificação multiclasse, atingindo desempenhos superiores de forma consistente em praticamente todas as arquiteturas testadas. Além disso, esta abordagem demonstrou maior estabilidade, apresentando melhorias mais graduais à medida que se aumentava a complexidade da rede. Por outro lado, a abordagem baseada na numeração da target, embora inicialmente apresente resultados bastante inferiores, revelou uma maior sensibilidade ao aumento da capacidade da rede e do número de epochs, registando melhorias mais acentuadas com arquiteturas mais profundas e mais tempo de treino. Este comportamento pode ser explicado pelo facto de esta estratégia exigir da rede uma aprendizagem mais precisa das relações entre os diferentes níveis de consumo. Por exemplo, uma determinada entrada pode ter Consumption MediumHigh (4) e a rede dar como output 3.4, que será arredondado para 3 (Medium), embora esteja muito perto do resultado correto. Isto fica ainda mais evidente se analisarmos a matriz de confusão dada pelo Scorer, correspondente à arquitetura de 4 layers (128, 64, 32, 16) com 5 epochs:

O efeito referido é mais acentuado para Medium, onde a rede apresenta dificuldades em determinar a “fronteira” entre os diversos níveis médios de consumo. De forma geral, os resultados

evidenciam também que as redes neurais desenvolvidas com Keras oferecem um grau de flexibilidade e configurabilidade substancialmente superior ao dos modelos anteriormente analisados, tornando a escolha da arquitetura, do método de representação dos dados e dos hiperparâmetros fatores decisivos para o desempenho final do modelo. Embora existisse margem para explorar arquiteturas mais complexas e processos de treino mais prolongados, essa exploração tornou-se limitada pelo elevado custo computacional associado ao treino destas redes.

7 Conclusão

Seguindo a metodologia SEMMA, começou-se por explorar o dataset de forma a identificar problemas de qualidade dos dados, padrões relevantes e possíveis transformações necessárias para a modelação. A análise realizada permitiu detectar valores em falta, outliers, inconsistências na target e features redundantes, conduzindo posteriormente à aplicação de diferentes estratégias de pré-processamento e transformação dos dados para resolver esses problemas.

Após a fase de modificação, foram desenvolvidos e avaliados cinco modelos de classificação distintos. De forma geral, os resultados mostram que os modelos baseados em árvores apresentaram o melhor desempenho neste problema, sobretudo os métodos de ensemble (Random Forest e Gradient Boosted Trees). Estes modelos revelaram também maior robustez relativamente às transformações aplicadas aos dados, sendo pouco sensíveis às diferentes estratégias de tratamento de missing values e outliers.

Por outro lado, as redes neurais demonstraram maior dependência da preparação dos dados. Em particular, a escolha do método de normalização e da representação das variáveis categóricas teve impacto significativo no desempenho obtido. O modelo desenvolvido com Keras destacou-se pela maior flexibilidade e capacidade de adaptação da arquitetura da rede, permitindo explorar diferentes formas de representação da target e diferentes configurações de treino. Ainda assim, os resultados obtidos permaneceram inferiores aos melhores modelos baseados em árvores.

De forma global, conclui-se que este problema é mais adequado a modelos baseados em árvores, capazes de capturar relações complexas entre as variáveis sem exigir um pré-processamento tão sensível como o requerido pelas redes neurais. Importa ainda referir que os resultados do modelo em Keras, ao contrário dos restantes modelos que foram avaliados com cross-validation, foram obtidos com uma simples partição hold-out, devido ao elevado custo computacional associado ao treino da rede neuronal.

[image1]: <data:image/png;base64,iVBORw0KGgoAAAANSUhEUgAAAl0AAAH7CAYAAAAAU8zE8AAC
[image2]: <data:image/png;base64,iVBORw0KGgoAAAANSUhEUgAAAbUAAAIICAYAAABSJGYDAAB
[image3]: <data:image/png;base64,iVBORw0KGgoAAAANSUhEUgAAAl0AAACoCAYAAAA4nkDRAAA
[image4]: <data:image/png;base64,iVBORw0KGgoAAAANSUhEUgAAAl0AAADHCAYAAADBPP1eAA
[image5]: <data:image/png;base64,iVBORw0KGgoAAAANSUhEUgAAAl0AAADpCAYAAAD8mpkYAA
[image6]: <data:image/png;base64,iVBORw0KGgoAAAANSUhEUgAAAl0AAAD4CAYAAAA0EEgmAAB
[image7]: <data:image/png;base64,iVBORw0KGgoAAAANSUhEUgAAAl0AAADJCAYAAAD7NpwuAAB
[image8]: <data:image/png;base64,iVBORw0KGgoAAAANSUhEUgAAAl0AAAE1CAYAAAA/EU74AAC

Método de normalização	Variáveis categóricas	Outliers	Missing values (números)	Missing values (categorias)
Min-max	numerar	manter	média	moda
Min-max	numerar	manter	média	fixo
Min-max	numerar	manter	mediana	moda
Min-max	numerar	manter	mediana	fixo
Min-max	numerar	ajustar	média	moda
Min-max	numerar	ajustar	média	fixo
Min-max	numerar	ajustar	mediana	moda
Min-max	numerar	ajustar	mediana	fixo
Min-max	One to Many	manter	média	moda
Min-max	One to Many	manter	média	fixo
Min-max	One to Many	manter	mediana	moda
Min-max	One to Many	manter	mediana	fixo
Min-max	One to Many	ajustar	média	moda
Min-max	One to Many	ajustar	média	fixo
Min-max	One to Many	ajustar	mediana	moda
Min-max	One to Many	ajustar	mediana	fixo
Z-score	numerar	manter	média	moda
Z-score	numerar	manter	média	fixo
Z-score	numerar	manter	mediana	moda
Z-score	numerar	manter	mediana	fixo
Z-score	numerar	ajustar	média	moda
Z-score	numerar	ajustar	média	fixo
Z-score	numerar	ajustar	mediana	moda
Z-score	numerar	ajustar	mediana	fixo
Z-score	One to Many	manter	média	moda
Z-score	One to Many	manter	média	fixo
Z-score	One to Many	manter	mediana	moda
Z-score	One to Many	manter	mediana	fixo
Z-score	One to Many	ajustar	média	moda
Z-score	One to Many	ajustar	média	fixo
Z-score	One to Many	ajustar	mediana	moda
Z-score	One to Many	ajustar	mediana	fixo

Nº iterations	Nº hidden layers	Nº hidden neurons per layer	Accuracy	Cohen's Kappa
100	1	10	0.886	0.854
100	1	20	0.885	0.853
100	1	30	0.882	0.848
100	1	40	0.885	0.853
100	2	10	0.843	0.798
100	2	20	0.87	0.833
100	2	30	0.879	0.845
100	2	40	0.867	0.83
100	3	10	0.829	0.78
100	3	20	0.83	0.782
100	3	30	0.841	0.797
100	3	40	0.856	0.816
200	1	10	0.9	0.873
200	1	20	0.899	0.871
200	1	30	0.896	0.867
200	1	40	0.893	0.863
200	2	10	0.866	0.828
200	2	20	0.889	0.859
200	2	30	0.879	0.845
200	2	40	0.869	0.833
200	3	10	0.857	0.817
200	3	20	0.846	0.802
200	3	30	0.862	0.823
200	3	40	0.86	0.821
300	1	10	0.903	0.875
300	1	20	0.9	0.872
300	1	30	0.895	0.865
300	1	40	0.888	0.856
300	2	10	0.884	0.851
300	2	20	0.889	0.858
300	2	30	0.877	0.843
300	2	40	0.865	0.828
300	3	10	0.872	0.837
300	3	20	0.853	0.812
300	3	30	0.862	0.823
300	3	40	0.858	0.819

Nº hidden layers	Nº units	Accuracy	Cohen's Kappa
1	16	0.576	0.457
2	16	8	0.639
2	32	16	0.724
3	32	16	8
3	64	32	16
4	128	64	32

Nº hidden layers	Nº units	Accuracy	Cohen's Kappa			
1	16	0.308	0.116			
2	16	8	0.297	0.102		
2	32	16	0.377	0.201		
3	32	16	8	0.36	0.182	
3	64	32	16	0.434	0.272	
4	128	64	32	16	0.513	0.371

Estratégia	Nº epochs	Accuracy	Cohen's Kappa
One to Many	5	0.779	0.718
One to Many	10	0.803	0.747
One to Many	20	0.842	0.798
Numerar	5	0.513	0.371
Numerar	10	0.639	0.533
Numerar	20	0.684	0.592

Prediction	Low	MediumLow	Medium	MediumHigh	High
Low	509	0	49	0	19
MediumLow	81	148	155	15	16
Medium	35	224	331	260	49
MediumHigh	8	124	85	345	87
High	0	7	0	6	67